

Trabajo Final - Food Store

Sistema de Gestión de Pedidos de Comida

Se propone desarrollar un ecommerce llamado Food Store, orientado a la venta de productos de un negocio de comidas mediante una aplicación web full stack. El sistema permitirá gestionar categorías, productos y pedidos, y contará con una gestión de usuarios con dos perfiles principales: ADMIN, con acceso al panel de administración para realizar operaciones CRUD y gestionar el estado de los pedidos, y USUARIO, que podrá registrarse, iniciar sesión, navegar el catálogo, seleccionar productos, administrar un carrito y confirmar compras, además de consultar el historial y el estado de sus pedidos.

El desarrollo deberá realizarse tomando como guía el backlog de épicas e historias de usuario, de modo que cada funcionalidad implementada cumpla con sus criterios de aceptación y represente el comportamiento esperado del sistema. Esto implica transformar las historias en pantallas y endpoints concretos, incorporando validaciones, manejo de errores y respuestas correctas, para que el producto final sea navegable, verificable y refleje los flujos principales descritos en el TPI.

A partir de esta base, el desarrollo se abordará como un proyecto full stack guiado por un backlog Scrum, donde las funcionalidades estarán organizadas en épicas e historias de usuario. En la práctica, esto significa que el estudiante deberá transformar cada historia en pantallas del frontend (según el rol ADMIN o USUARIO) y en endpoints de la API REST del backend, respetando sus criterios de aceptación, validaciones y reglas de negocio. Por ejemplo, el rol ADMIN deberá contar con un panel para realizar operaciones CRUD sobre categorías y productos, además de gestionar pedidos y sus estados; mientras que el rol USUARIO deberá poder registrarse, iniciar sesión, navegar el catálogo, operar un carrito persistente, confirmar compras y consultar el historial/estado de sus pedidos. El objetivo es que el producto final sea navegable y verificable, y que los flujos principales del sistema puedan evaluarse claramente a partir de lo definido en las historias.

Sistema de Gestión de Pedidos de Comida	1
 Información del Proyecto	3
 Objetivos del Proyecto	3
 Estructura del Proyecto	3
Estructura Frontend (ejemplo de proyecto).....	3
Estructura Backend (ejemplo de proyecto).....	4
Sistema de Autenticación y Autorización	4
Roles y Permisos	5
Modelo UML	5
Funcionalidades por Módulo	6
1. Módulo de Autenticación.....	6
2. Módulo de Cliente - Store	7

3. Módulo de Cliente - Mis Pedidos	8
4. Módulo de Administración.....	9
 Diseño y UX	11
Pantallas de Referencia.....	11
Flujos de Usuario	16
Flujo de Compra (Cliente)	16
Flujo de Gestión de Producto (Admin).....	16
Flujo de Gestión de Pedido (Admin)	17
Consideraciones Importantes.....	17
 Seguridad	17
Historias de Usuario (Backlog Scrum).....	17
Guía rápida: qué funcionalidades programar	17
☒ Entrega del Proyecto.....	18
Contenido de la Entrega	18
Método de Entrega	18
Anexo – Historias de Usuario.....	18
Historias de Usuario - Sistema de Gestión de Pedidos.....	18
Roles del Sistema	20
Épicas del Proyecto	21
EP-01: Gestion de Categorías.....	21
EP-02: Gestion de Usuarios	21
EP-03: Gestión de Productos.....	21
EP-04: Gestión de Pedidos	22
EP-05: Infraestructura y Arquitectura	22
Historias de Usuario por épica	22
EP-01: Gestion de Categorías.....	22
EP-02: Gestion de Usuarios.....	33
EP-03: Gestión de Productos.....	43
EP-04: Gestión de Pedidos	54
EP-05: Infraestructura y Arquitectura	67
Matriz de Trazabilidad	78
Historias de Usuario por Epica	78
Dependencias entre Historias	78
Priorización del Backlog.....	79
Sprint 1 - Fundamentos (Semana 1-2)	79
Sprint 2 - Usuarios y Productos (Semana 3-4).....	80
Sprint 3 - Pedidos (Semana 5-6).....	80
Definición de Completado (DoD).....	80
Código	81
Testing	81
Documentación.....	81

Revisión.....	81
Funcionalidad.....	81



Información del Proyecto

Nombre: Food Store - Sistema de Gestión de Pedidos de Comida

Tecnologías:

- **Nombre:** Food Store - Sistema de Gestión de Pedidos de Comida
- **Tecnologías Frontend:** TypeScript, Vite, HTML5, CSS3, Tailwind CSS
- **Tecnologías Backend:** Spring Boot 3.x, Java 17+, PostgreSQL/MySQL
- **Autenticación:** Gestión básica con localStorage (sólo fines educativos)



Objetivos del Proyecto

Desarrollar una aplicación web full stack completa para la gestión de un negocio de comidas, que permita:

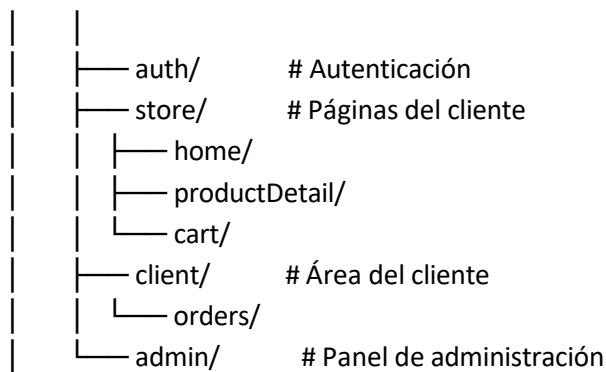
1. **A los administradores:** Gestionar categorías, productos y pedidos
2. **A los clientes:** Navegar productos, realizar compras y seguir sus pedidos
3. **Sistema de carrito:** Funcional con persistencia en localStorage
4. **Integración completa:** Conexión Frontend-Backend mediante REST API



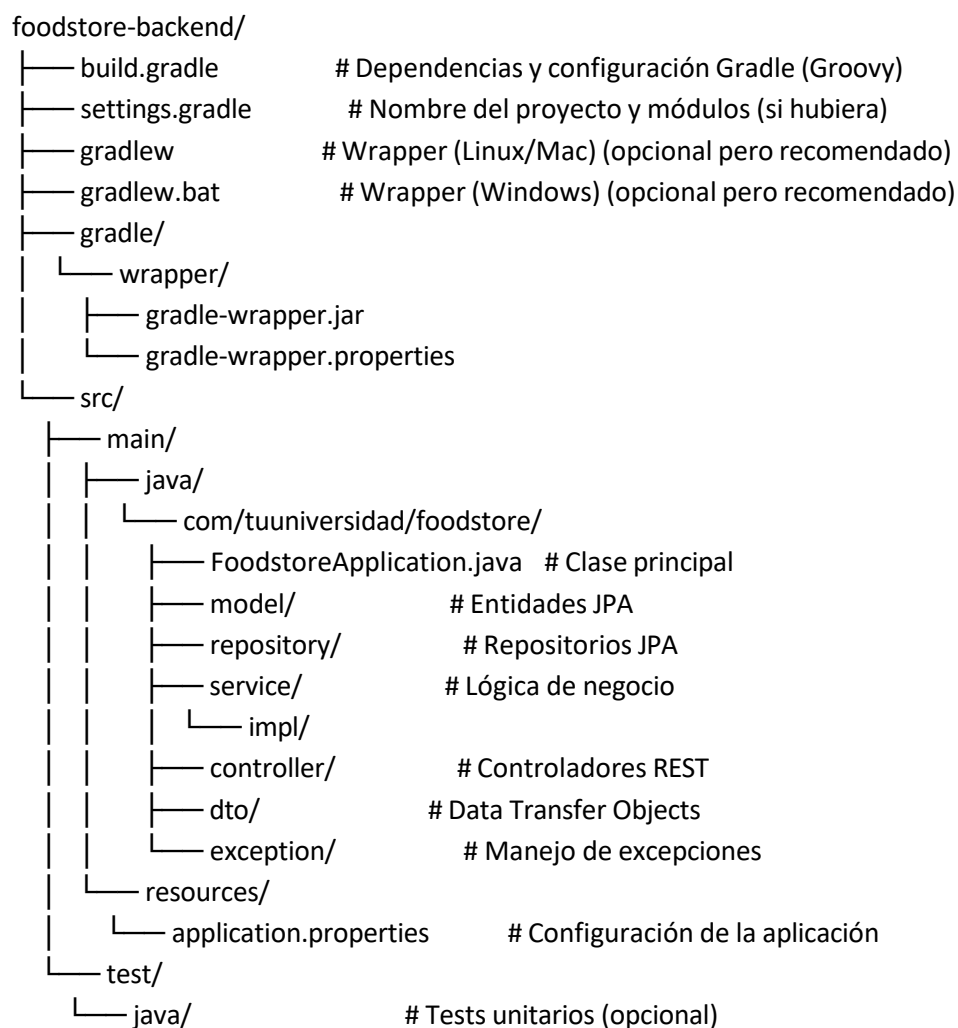
Estructura del Proyecto

Estructura Frontend (ejemplo de proyecto)

```
final-prog3/
├── index.html          # Redirección a login
├── package.json        # Dependencias y scripts
├── tsconfig.json       # Configuración TypeScript
├── vite.config.ts      # Configuración Vite
├── src/
│   ├── main.ts         # Punto de entrada (vacío)
│   ├── style.css       # Estilos globales
│   ├── types/          # Definiciones de tipos TypeScript
│   ├── utils/          # Utilidades y helpers
│   └── pages/          # Páginas de la aplicación
```



Estructura Backend (ejemplo de proyecto)



Sistema de Autenticación y Autorización

Flujo de Autenticación

1. Login/Registro:

- Usuario ingresa credenciales
- Frontend envía POST a `/api/auth/login` o `/api/auth/register`
- Backend valida credenciales
- Si es exitoso, retorna datos del usuario
- Frontend guarda datos en `localStorage`
- Redirecciona según el rol

2. Validación de Sesión:

- Cada página protegida verifica `localStorage`
- Si no hay sesión, redirecciona a login
- Valida permisos según el rol

4. Cierre de Sesión:

- Limpia `localStorage`
- Redirecciona al login

Roles y Permisos

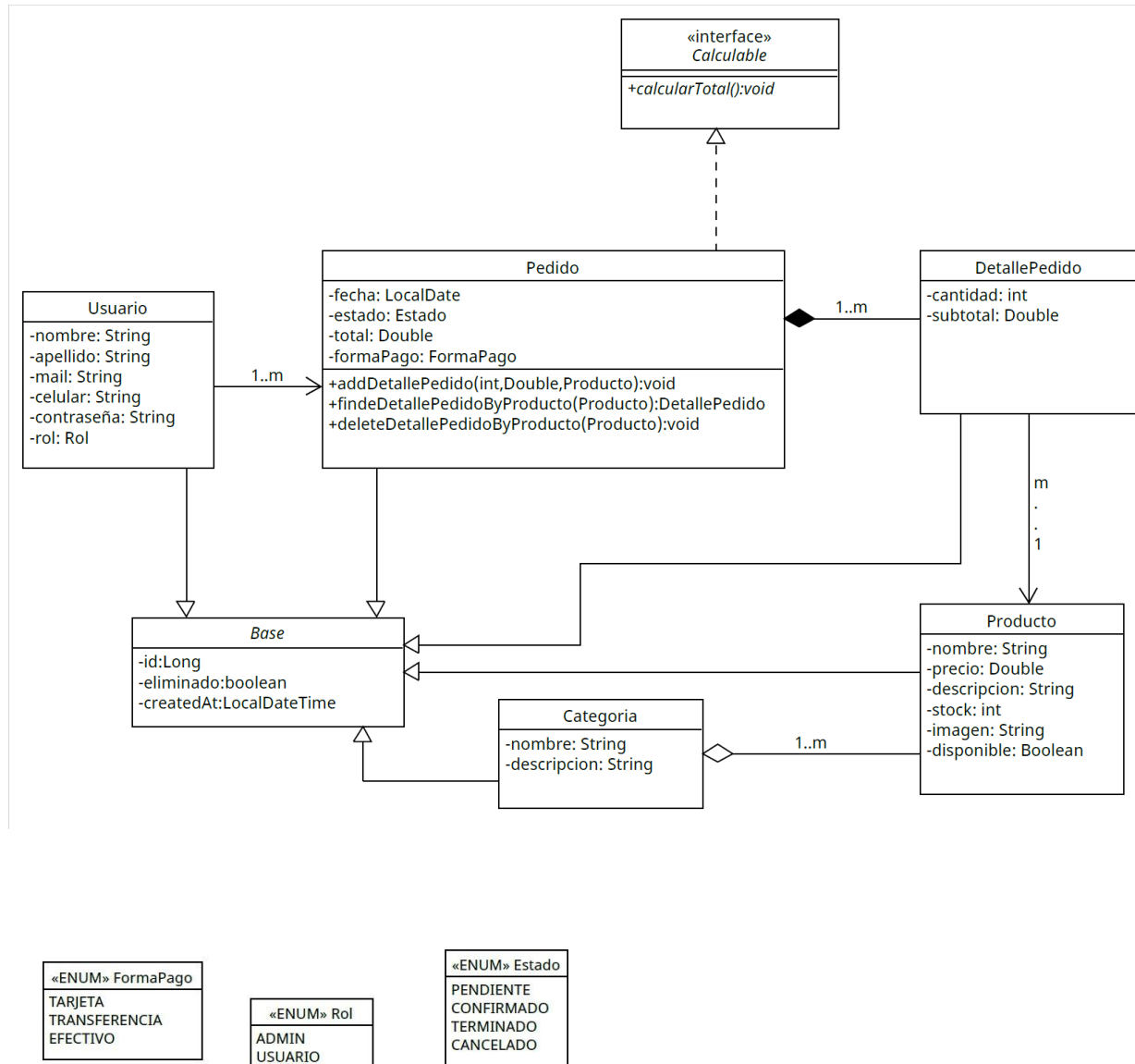
Administrador (Admin)

- ☒ Acceso completo al panel de administración
- ☒ Gestión de categorías (CRUD)
- ☒ Gestión de productos (CRUD)
- ☒ Gestión de pedidos (ver todos, actualizar estado)

Cliente (Usuario)

- ☒ Ver catálogo de productos
- ☒ Filtrar por categorías
- ☒ Buscar productos
- ☒ Ver detalle de productos
- ☒ Agregar productos al carrito
- ☒ Gestionar carrito (agregar, quitar, modificar cantidades)
- ☒ Realizar pedidos
- ☒ Ver historial de pedidos propios
- ☒ Ver detalle y estado de sus pedidos
- ☒ NO tiene acceso al panel de administración

Modelo UML



Funcionalidades por Módulo

1. Módulo de Autenticación

Login (/src/pages/auth/login/)

Funcionalidad:

- Formulario con email y contraseña
- Validación de campos requeridos
- Conexión con API POST /api/auth/login
- Manejo de errores de autenticación

- Redirección según rol del usuario

Registro (/src/pages/auth/register/)

Funcionalidad:

- Formulario con nombre, email y contraseña
- Validación de campos (email válido, contraseña mínimo 6 caracteres)
- Conexión con API POST /api/auth/register
- Solo se pueden registrar clientes
- Auto-login después del registro

2. Módulo de Cliente - Store

Home / Catálogo (/src/pages/store/home/)

Características:

- Sidebar con categorías (GET /api/categories)
- Búsqueda en tiempo real
- Filtros:
 - Por categoría
 - Ordenamiento (nombre A-Z, Z-A, precio ascendente/descendente)
- Grid de productos (GET /api/products) con:
 - Imagen
 - Nombre
 - Descripción
 - Precio
 - Badge de disponibilidad
 - Click para ir al detalle
- Badge del carrito con cantidad de ítems
- Contador de productos encontrados
- Toggle del sidebar para mobile

Detalle de Producto (/src/pages/store/productDetail/)

Características:

- GET /api/products/{id}
- Imagen grande del producto
- Información completa:
 - Nombre
 - Descripción
 - Precio
 - Stock disponible
 - Estado (disponible/no disponible)
- Selector de cantidad con validación de stock
- Botón "Agregar al Carrito"
- Mensaje de confirmación
- Botón "Volver"

- **Validaciones:**
 - No permite agregar si no hay stock
 - No permite cantidad mayor al stock disponible
 - No permite agregar productos inactivos

Carrito de Compras (/src/pages/store/cart/)

Características:

- Lista de productos en el carrito con:
 - Imagen
 - Nombre
 - Precio unitario
 - Controles de cantidad (+/-)
 - Precio total por producto
 - Botón eliminar
- Resumen del pedido:
 - Subtotal
 - Total
- Botones:
 - "Vaciar Carrito"
- Modal de checkout con formulario:
 - Teléfono (requerido)
- Al confirmar: POST /api/orders
- **Validaciones:**
 - Stock disponible al modificar cantidad
 - Campos requeridos en checkout
- Estado vacío con mensaje y botón a la tienda

Persistencia:

- Carrito guardado en localStorage
- Persiste entre sesiones
- Se limpia al confirmar pedido

3. Módulo de Cliente - Mis Pedidos

Historial de Pedidos (/src/pages/client/orders/)

Características:

- GET /api/orders (solo pedidos del usuario)
- Lista de pedidos del usuario autenticado
- Tarjetas de pedido mostrando:

- Número de pedido
- Fecha y hora
- Estado con badge de color
- Resumen de productos (primeros 3 + contador)
- Total del pedido
- Click en pedido abre modal con detalle completo
- Modal de detalle muestra:
 - Estado con icono visual
 - Información de entrega
 - Lista completa de productos
 - Desglose de costos (subtotal, envío, total)
 - Mensaje según estado del pedido
- Estado vacío si no hay pedidos

Estados de Pedido:

- `pending:` ⌚ Pendiente
- `processing:` 🔍 En Preparación
- `completed:` ☑ Entregado
- `cancelled:` ✕ Cancelado

4. Módulo de Administración

Dashboard (/src/pages/admin/adminHome/)

Características:

- Sidebar de navegación administrativa
- 4 tarjetas con estadísticas:
 - Total de categorías (GET /api/categories)
 - Total de productos (GET /api/products)
 - Total de pedidos (GET /api/orders)
 - Productos disponibles
- Panel de resumen detallado:
 - Categorías activas
 - Productos activos/inactivos
 - Pedidos por estado
- Enlaces directos a cada módulo

Gestión de Categorías (/src/pages/admin/categories/)

Características:

- GET /api/categories

- Tabla con todas las categorías:
 - ID
 - Imagen (thumbnail)
 - Nombre
 - Descripción
 - Acciones (Editar/Eliminar)
- Botón "Nueva Categoría"
- Modal para crear/editar:
 - POST /api/categories (crear)
 - PUT /api/categories/{id} (editar)
 - Nombre (requerido)
 - Descripción (requerida)
 - URL de imagen (requerida)
- **Validaciones:**
 - Campos requeridos
 - URL válida para imagen
- Confirmación al eliminar (DELETE /api/categories/{id})

Gestión de Productos (/src/pages/admin/products/)

Características:

- GET /api/products
- Tabla con todos los productos:
 - ID
 - Imagen (thumbnail)
 - Nombre
 - Descripción
 - Precio
 - Categoría (nombre)
 - Stock
 - Estado (disponible/no disponible)
 - Acciones (Editar/Eliminar)
- Botón "Nuevo Producto"
- Modal para crear/editar:
 - POST /api/products (crear)
 - PUT /api/products/{id} (editar)
 - Nombre (requerido)
 - Descripción (requerida)
 - Precio (requerido, > 0)
 - Stock (requerido, >= 0)
 - Categoría (select con categorías disponibles)
 - URL de imagen (requerida)
 - Checkbox "Producto disponible"

- **Validaciones:**
 - Todos los campos requeridos
 - Precio y stock numéricos válidos
 - Categoría debe existir
- Confirmación al eliminar (DELETE /api/products/{id})

Gestión de Pedidos Admin (/src/pages/admin/orders/)

Características:

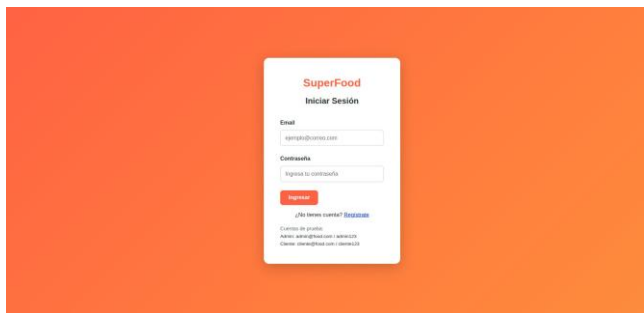
- GET /api/orders (todos los pedidos)
- Filtro por estado de pedido
- Lista de TODOS los pedidos (no solo del usuario)
- Tarjetas de pedido mostrando:
 - Número de pedido
 - Nombre del cliente
 - Fecha y hora
 - Estado con badge
 - Cantidad de productos
 - Total
- Click en pedido abre modal con detalle
- Modal muestra:
 - Información del cliente
 - Dirección y teléfono
 - Método de pago
 - Notas del cliente
 - Lista completa de productos
 - Desglose de costos
 - **Select para cambiar estado del pedido**
 - Botón "Actualizar Estado" (PATCH /api/orders/{id}/status)
- Ordenados por fecha (más recientes primero)



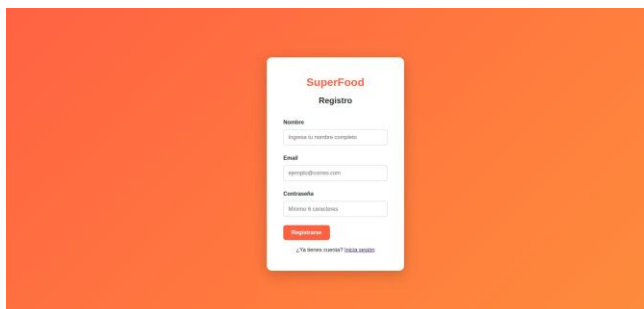
Diseño y UX

Pantallas de Referencia

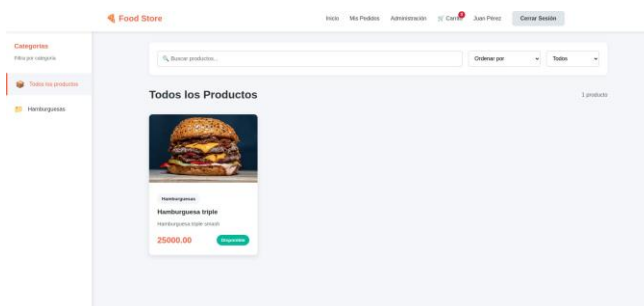
Login



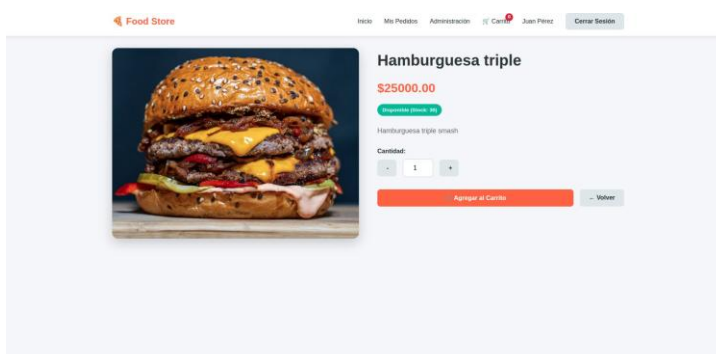
Registro



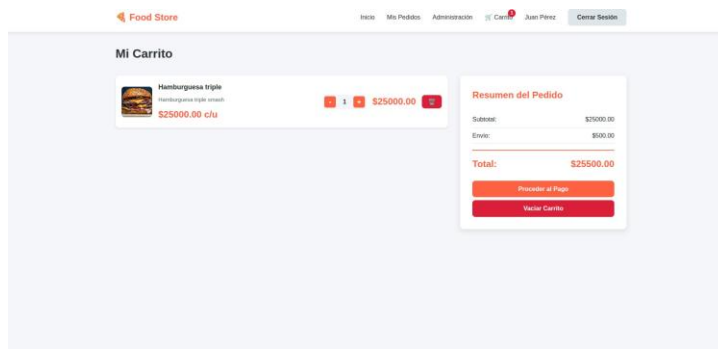
Vista Home Store



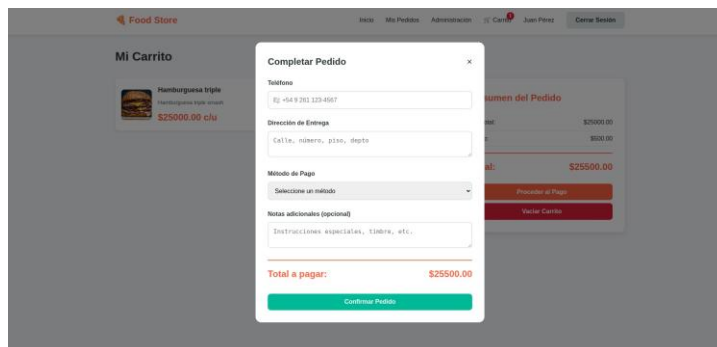
Vista Detalle de Producto



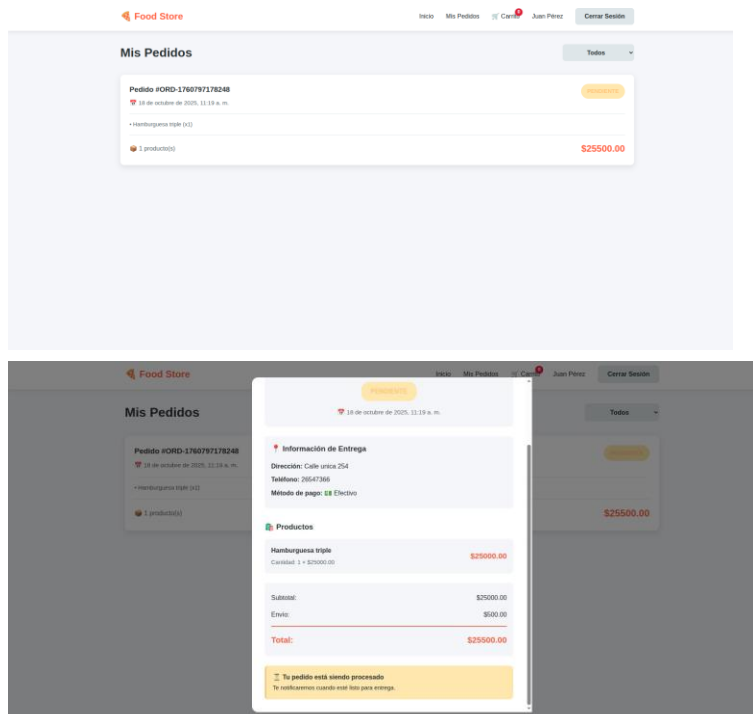
Vista de Carrito



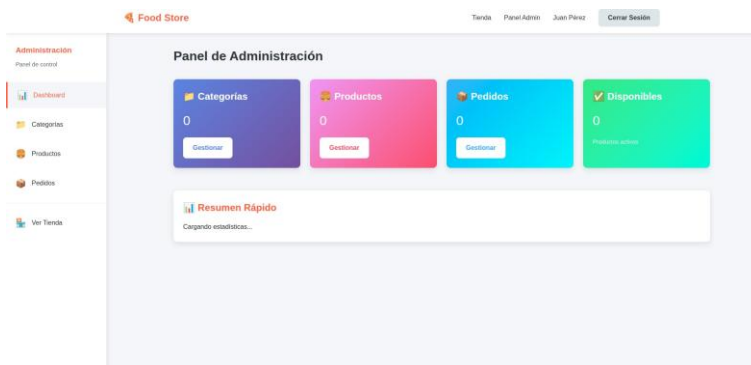
Confirmación de Pedido



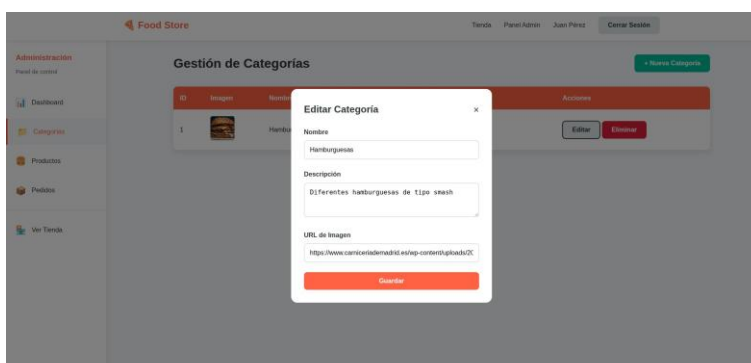
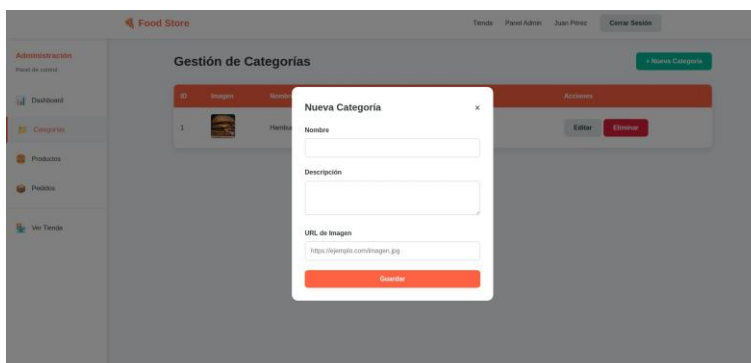
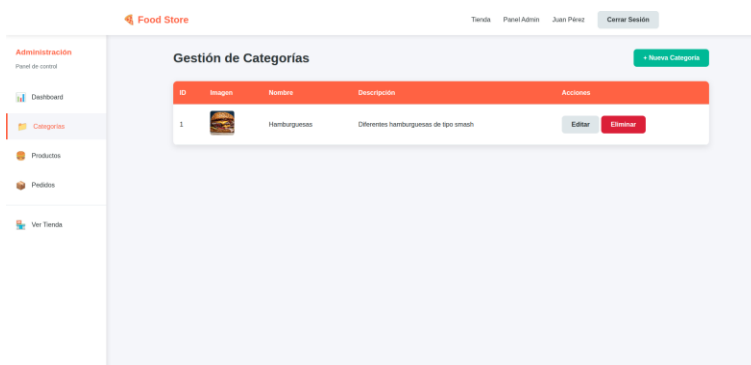
Pedidos del Cliente



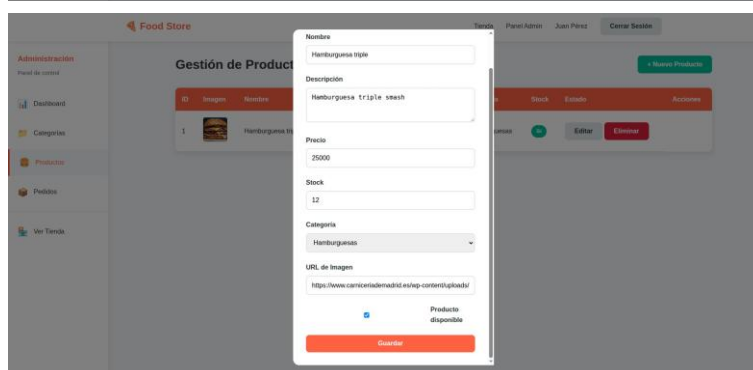
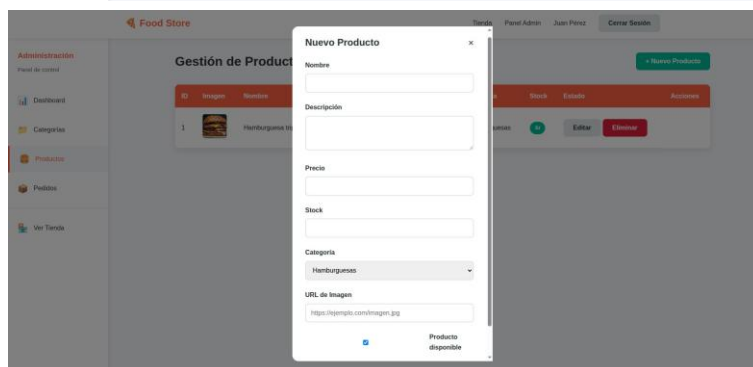
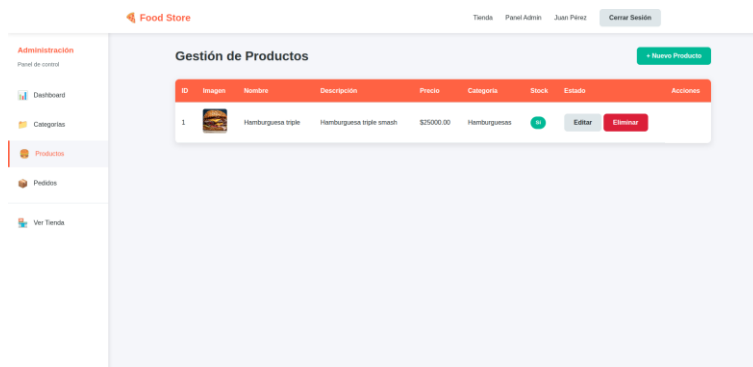
Home Admin



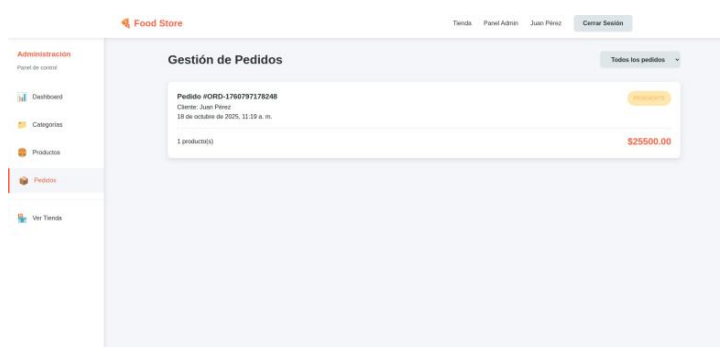
CRUD Categorías

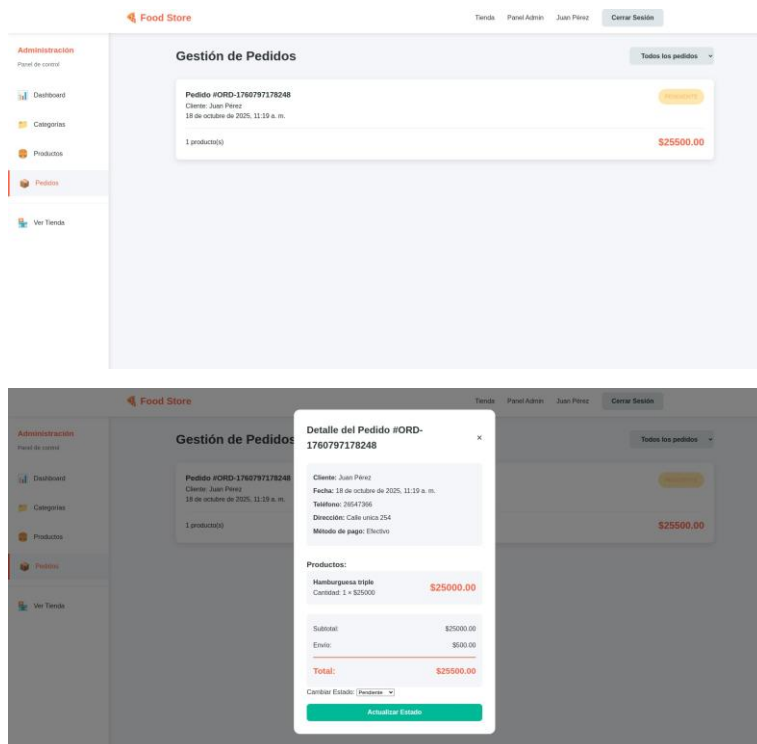


CRUD Productos



Gestión de Pedidos





Flujos de Usuario

Flujo de Compra (Cliente)

1. Usuario se autentica
2. Navega por el catálogo (puede filtrar/buscar)
3. Click en producto → Ver detalle
4. Selecciona cantidad → Agregar al carrito
5. Continúa comprando o va al carrito
6. En el carrito: revisa productos, modifica cantidades
7. Click "Proceder al pago"
8. Completa formulario de checkout
9. Confirma pedido (POST /api/orders)
10. Ver mensaje de éxito
11. Es redirigido a "Mis Pedidos"
12. Puede ver estado y detalle del pedido

Flujo de Gestión de Producto (Admin)

1. Admin se autentica
2. Va a "Panel Admin" → "Productos"
3. Click en "Nuevo Producto"

4. Completa formulario
5. Selecciona categoría del dropdown
6. Marca checkbox si está disponible
7. Guarda producto (POST /api/products)
8. Ve el producto en la tabla
9. Puede editar/eliminar cuando necesite

Flujo de Gestión de Pedido (Admin)

1. Admin va a "Pedidos"
2. Ver lista de todos los pedidos (GET /api/orders)
3. Puede filtrar por estado
4. Click en pedido → Ver detalle completo
5. Cambia estado en el select
6. Click "Actualizar Estado" (PATCH /api/orders/{id}/status)
7. Cliente ve el cambio en "Mis Pedidos"

Consideraciones Importantes

Seguridad

IMPORTANTE: Este proyecto NO implementa seguridad real:

- Las contraseñas se almacenan encriptada en SHA-256
- No hay tokens JWT
- La validación de rol es solo frontend
- localStorage es fácilmente modificable
- **SOLO para fines educativos**

Historias de Usuario (Backlog Scrum)

Esta sección integra el backlog del proyecto en formato de historias de usuario. Su objetivo es que, al leer el TPI, el estudiante tenga una idea clara de qué se espera programar en cada funcionalidad, incluyendo validaciones y comportamiento esperado.

Guía rápida: qué funcionalidades programar

EP-01 – Gestión de Categorías: CRUD completo de categorías con validaciones y soft delete.

EP-02 – Gestión de Usuarios: Registro/autenticación y administración, email único, password encriptada y DTOs sin password.

EP-03 – Gestión de Productos: CRUD de productos con control de stock/precio, asociación a categoría y filtros.

EP-04 – Gestión de Pedidos: Creación/consulta de pedidos con detalle, cálculo de totales, reducción de stock y transacciones.

EP-05 – Infraestructura y Arquitectura: Capas, DTOs, excepciones centralizadas, auditoría y buenas prácticas.

Entrega del Proyecto

Contenido de la Entrega

1. Código Fuente

- Repositorio GitHub con todo el código del frontend y el backend.
- Archivo README.md en la raíz del repositorio con instrucciones claras de instalación, configuración de la base de datos y comandos para ejecutar el proyecto.

2. Documentación Académica y Técnica (Formato PDF)

El documento debe estar correctamente ordenado, con las hojas numeradas y mantener un formato profesional. Debe incluir:

- **Carátula:** Nombre de la institución, carrera, nombre de la materia, título del proyecto ("Food Store"), datos del estudiante y fecha de entrega.
- **Índice:** Tabla de contenidos estructurada con los números de página correspondientes para facilitar la navegación.
- **Marco Teórico:** Breve descripción de las tecnologías utilizadas (Spring Boot 3.x, Java, Vite, etc.) y conceptos clave aplicados.
- **Decisiones Técnicas y Arquitectura:** Explicación de cómo se abordó la solución, por qué se aplicaron ciertos patrones (por ejemplo, el uso de DTOs, manejo global de excepciones), y capturas de pantalla de las interfaces principales del sistema.
- **Dificultades y Soluciones:** Un breve apartado comentando los principales obstáculos técnicos que surgieron durante los sprints y cómo lograron resolverlos.
- **Bibliografía / Webgrafía:** Enlaces a la documentación oficial, repositorios o recursos utilizados como apoyo durante el desarrollo.

3. Video Demostración

- Grabación de 10 a 15 minutos mostrando el funcionamiento del sistema ya levantado.
- Se debe recorrer obligatoriamente el flujo del cliente (registro, navegación, uso del carrito, confirmación de pedido) y el flujo del administrador (gestión del catálogo y actualización de estados de pedidos).

Método de Entrega:

La entrega del proyecto deberá realizarse **exclusivamente en formato .zip**. El archivo comprimido deberá contener obligatoriamente:

- El **código fuente completo desarrollado**.
- El **documento en formato PDF** confeccionado según las consignas establecidas.

Asimismo, dentro del archivo **README.md** del repositorio se deberá incluir obligatoriamente:

- El **enlace al video demostrativo**, asegurando que cuente con permisos públicos de visualización.
- El **enlace a la documentación en PDF**, o bien el archivo PDF subido directamente en la raíz del repositorio.

No se considerarán entregas incompletas o que no respeten el formato indicado.

Anexo – Historias de Usuario

Se incorpora el contenido completo del documento de Historias de Usuario: épicas, historias, criterios de aceptación, reglas de negocio y tareas técnicas.

Historias de Usuario - Sistema de Gestión de Pedidos

Información del Proyecto

Campo	Valor
Proyecto	TPI Programacion 3 - Sistema de Gestión de Pedidos
Versión	1.0.0
Fecha de Creación	Diciembre 2024
Metodología	Agile/Scrum
Stack Tecnológico	Spring Boot 3.4.12, Java 21, JPA/Hibernate, H2/PostgreSQL

Tabla de Contenidos

[Glosario de Términos](#glosario-de-terminos)

[Roles del Sistema](#roles-del-sistema)

[Épicas del Proyecto](#epicas-del-proyecto)

[Historias de Usuario](#historias-de-usuario)

[EP-01: Gestion de Categorías](#ep-01-gestion-de-categorias)

[EP-02: Gestion de Usuarios](#ep-02-gestion-de-usuarios)

[EP-03: Gestión de Productos](#ep-03-gestion-de-productos)

[EP-04: Gestión de Pedidos](#ep-04-gestion-de-pedidos)

[EP-05: Infraestructura y Arquitectura](#ep-05-infraestructura-y-arquitectura)

[Matriz de Trazabilidad](#matriz-de-trazabilidad)

[Priorización del Backlog](#priorizacion-del-backlog)

[Definición de Completado (DoD)](#definicion-de-completado-dod)

Glosario de Términos

Termino	Definicion
---------	------------

-----	-----
-------	-------

API REST	Interfaz de programación de aplicaciones que utiliza el protocolo HTTP para operaciones CRUD
---------------------	--

Soft Delete	Eliminación lógica donde el registro se marca como eliminado pero permanece en la base de datos
------------------------	---

DTO	Data Transfer Object - Objeto para transferir datos entre capas de la aplicación
----------------	--

JWT	JSON Web Token - Estandar para autenticación (consideración futura)
----------------	---

CRUD	Create, Read, Update, Delete - Operaciones básicas de persistencia
-----------------	--

Endpoint	Punto de acceso a un recurso de la API
---------------------	--

Transacción	Conjunto de operaciones que se ejecutan como unidad atómica
------------------------	---

Validación	Proceso de verificar que los datos cumplen con las reglas de negocio
-----------------------	--

Roles del Sistema

ROL-01: Administrador (ADMIN)

****Descripción:**** Usuario con acceso completo al sistema. Puede gestionar todas las entidades, usuarios y configuraciones.

****Permisos:****

Gestión completa de categorías (CRUD)

Gestión completa de productos (CRUD)

Gestión completa de usuarios (CRUD)

Gestión completa de pedidos (CRUD)

Cambio de estados de pedidos

Acceso a reportes y estadísticas

ROL-02: Usuario (USUARIO)

****Descripción:**** Usuario regular del sistema. Puede realizar compras y gestionar sus propios pedidos.

****Permisos:****

Visualización de categorías

Visualización de productos

Gestión de su perfil

Creación y visualización de sus pedidos

Cancelación de sus pedidos (en estado PENDIENTE)

ROL-03: Sistema (SYSTEM)

****Descripción:**** Procesos automáticos del sistema.

****Responsabilidades:****

Carga inicial de datos

Auditoria automática (timestamps)

Validaciones automáticas

Control de concurrencia

Épicas del Proyecto

EP-01: Gestion de Categorías

****Descripción:**** Funcionalidades relacionadas con la administración de categorías de productos.

****Objetivo de Negocio:**** Permitir la organización y clasificación de productos para facilitar la navegación y búsqueda.

****Valor:**** Alto

****Complejidad:**** Baja

EP-02: Gestion de Usuarios

****Descripción:**** Funcionalidades relacionadas con el registro, autenticación y administración de usuarios.

****Objetivo de Negocio:**** Gestionar la identidad de los usuarios y controlar el acceso al sistema.

****Valor:**** Crítico

****Complejidad:**** Media

EP-03: Gestión de Productos

****Descripción:**** Funcionalidades relacionadas con la administración del catálogo de productos.

****Objetivo de Negocio:**** Mantener un catálogo de productos actualizado con control de inventario.

****Valor:**** Alto

****Complejidad:**** Media

EP-04: Gestión de Pedidos

****Descripción:**** Funcionalidades relacionadas con la creación, seguimiento y gestión de pedidos.

****Objetivo de Negocio:**** Permitir a los usuarios realizar compras y hacer seguimiento de sus pedidos.

****Valor:**** Crítico

****Complejidad:**** Alta

EP-05: Infraestructura y Arquitectura

****Descripción:**** Funcionalidades transversales de arquitectura, seguridad y mantenibilidad.

****Objetivo de Negocio:**** Garantizar la calidad, seguridad y escalabilidad del sistema.

****Valor:**** Crítico

****Complejidad:**** Alta

Historias de Usuario por épica

EP-01: Gestion de Categorías

HU-001: Crear Categoría

Información General

Campo	Valor
-----	-----
ID	HU-001
Epica	EP-01: Gestion de Categorías
Título	Crear Categoría
Prioridad	Alta
Puntos de Historia	3
Sprint	1

Historia de Usuario

****Como**** Administrador del sistema

****Quiero**** poder crear nuevas categorías de productos

****Para**** organizar el catálogo de productos de manera estructurada y facilitar la navegación de los usuarios

Descripción Detallada

El administrador necesita la capacidad de crear categorías que servirán para clasificar los productos del sistema. Cada categoría debe tener un nombre único y descriptivo, y opcionalmente una descripción que explique el tipo de productos que contiene.

Criterios de Aceptación

Escenario 1: Creación exitosa de categoria

Dado que soy un administrador autenticado

Y tengo los datos validos de una categoría:

nombre	"Electrónica"	
descripcion	"Productos electrónicos y gadgets"	

Cuando envía una petición POST a /categoria

Entonces el sistema debe retornar código 201 Created

Y el cuerpo de respuesta debe contener el ID generado

Y el cuerpo debe incluir los datos de la categoría creada

Y la categoría debe persistir en la base de datos

Escenario 2: Error por nombre vacío

Dado que soy un administrador autenticado

Y el campo nombre esta vacio o solo contiene espacios

Cuando envía una petición POST a /categoria

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El nombre es obligatorio"

Escenario 3: Error por nombre muy corto

Dado que soy un administrador autenticado

Y el campo nombre tiene menos de 2 caracteres

Cuando envía una petición POST a /categoria

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El nombre debe tener entre 2 y 100 caracteres"

Escenario 4: Error por nombre muy largo

Dado que soy un administrador autenticado

Y el campo nombre tiene más de 100 caracteres

Cuando envía una petición POST a /categoria

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El nombre debe tener entre 2 y 100 caracteres"

Escenario 5: Error por descripción muy larga

Dado que soy un administrador autenticado

Y el campo descripción tiene mas de 500 caracteres

Cuando envía una petición POST a /categoria

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "La descripción no puede exceder 500 caracteres"

Datos de Entrada (Request)

```
{  
  "nombre": "string (obligatorio, 2-100 caracteres)",  
  "descripcion": "string (opcional, máx 500 caracteres)"  
}
```

Datos de Salida (Response)

```
{  
  "id": "long (autogenerado)",  
  "nombre": "string",  
  "descripcion": "string"  
}
```

Reglas de Negocio

| ID | Regla |

| ---- | ----- |

- | RN-001-01 | El nombre de la categoria es obligatorio |
- | RN-001-02 | El nombre debe tener entre 2 y 100 caracteres |
- | RN-001-03 | La descripción es opcional |
- | RN-001-04 | La descripcion no puede exceder 500 caracteres |
- | RN-001-05 | El ID se genera automaticamente |
- | RN-001-06 | Los campos createdAt y updatedAt se generan automáticamente |

Tareas Técnicas

- [] Crear entidad `Categoria` con campos id, nombre, descripcion
- [] Crear DTO `CategoriaCreate` con validaciones Jakarta
- [] Crear DTO `CategoriaDto` para respuesta
- [] Implementar método `save()` en `CategoriaService`
- [] Implementar endpoint POST en `CategoriaController`
- [] Escribir tests unitarios del servicio
- [] Escribir tests de integración del endpoint
- [] Documentar endpoint en Swagger

Notas Técnicas

- Usar `@NotBlank` para validar nombre obligatorio
- Usar `@Size` para validar longitud de campos
- La entidad debe extender de `Base` para heredar id y auditoría
- Usar Java Records para los DTOs

HU-002: Listar Categorías

Información General

- | Campo | Valor |
- | ----- | ----- |
- | **ID** | HU-002 |

****Epica****	EP-01: Gestion de Categorías
****Título****	Listar Categorías
****Prioridad****	Alta
****Puntos de Historia****	2
****Sprint****	1

Historia de Usuario

****Como**** Usuario del sistema

****Quiero**** ver todas las categorías disponibles

****Para**** poder navegar y filtrar productos por categoría

Descripción Detallada

Los usuarios necesitan visualizar todas las categorías activas del sistema para poder explorar el catálogo de productos. Solo se deben mostrar las categorías que no han sido eliminadas (soft delete).

Criterios de Aceptación

Escenario 1: Listar categorías existentes

Dado que existen categorías activas en el sistema

Cuando envío una petición GET a /categoria

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array de categorías

Y cada categoría debe tener id, nombre y descripción

Y no debe incluir categorías eliminadas

Escenario 2: Lista vacía

Dado que no existen categorías activas en el sistema

Cuando envío una petición GET a /categoria

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array vacío []

Datos de Salida (Response)

[

```
{  
  "id": 1,  
  "nombre": "Electrónica",  
  "descripcion": "Productos electrónicos"  
},  
{  
  "id": 2,  
  "nombre": "Ropa",  
  "descripción": "Prendas de vestir"  
}  
]
```

Reglas de Negocio

ID	Regla
----	-----
RN-002-01	Solo se muestran categorias con eliminado = false
RN-002-02	El listado no requiere autenticacion
RN-002-03	El orden de las categorias no esta definido

Tareas Técnicas

- [] Implementar método `findAll()` en `CategoriaService`
- [] Usar `findAllByEliminadoFalse()` del `BaseRepository`
- [] Implementar endpoint GET en `CategoriaController`
- [] Mapear entidades a DTOs
- [] Escribir tests unitarios
- [] Escribir tests de integración

HU-003: Obtener Categoría por ID

Información General

```
| Campo | Valor |  
|-----|-----|  
| **ID** | HU-003 |  
| **Epica** | EP-01: Gestion de Categorías |  
| **Título** | Obtener Categoría por ID |  
| **Prioridad** | Media |  
| **Puntos de Historia** | 2 |  
| **Sprint** | 1 |
```

Historia de Usuario

****Como**** Usuario del sistema

****Quiero**** obtener los detalles de una categoria especifica

****Para**** ver la informacion completa de la categoría

Criterios de Aceptación

Escenario 1: Categoría encontrada

Dado que existe una categoría con id 1
Y la categoría no esta eliminada
Cuando envio una peticion GET a /categoria/1
Entonces el sistema debe retornar código 200 OK
Y el cuerpo debe contener los datos de la categoría

Escenario 2: Categoría no encontrada

Dado que no existe una categoría con id 999
Cuando envio una peticion GET a /categoria/999
Entonces el sistema debe retornar código 404 Not Found
Y el mensaje debe indicar "Entidad con id 999 no encontrado"

Escenario 3: Categoría eliminada

Dado que existe una categoría con id 1
Y la categoría esta marcada como eliminada

Cuando envío una petición GET a /categoria/1

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

ID	Regla
----	-------

----	-----
------	-------

RN-003-01	Solo se puede obtener categorias con eliminado = false
-----------	--

RN-003-02	Si la categoria no existe, retornar 404
-----------	---

Tareas Técnicas

[] Implementar método `findById()` en `CategoriaService`

[] Usar `findByIdOrThrow()` del BaseRepository

[] Implementar endpoint GET /{id} en `CategoriaController`

[] Manejar `ResourceNotFoundException`

HU-004: Actualizar Categoría

Información General

Campo	Valor
-------	-------

-----	-----
-------	-------

ID	HU-004
-----------	--------

Epica	EP-01: Gestion de Categorias
--------------	------------------------------

Título	Actualizar Categoría
---------------	----------------------

Prioridad	Media
------------------	-------

Puntos de Historia	3
---------------------------	---

Sprint	1
---------------	---

Historia de Usuario

Como Administrador del sistema

Quiero poder modificar los datos de una categoría existente

****Para**** mantener actualizada la información del catálogo

Criterios de Aceptación

Escenario 1: Actualización exitosa completa

Dado que soy un administrador autenticado
Y existe una categoría con id 1
Y envío datos válidos para actualizar
Cuando envío una petición PUT a /categoria/1
Entonces el sistema debe retornar código 200 OK
Y el cuerpo debe contener los datos actualizados
Y el campo updatedAt debe actualizarse

Escenario 2: Actualización parcial (solo nombre)

Dado que soy un administrador autenticado
Y existe una categoría con id 1 con descripción "Descripción original"
Y envío solo el campo nombre en el request
Cuando envío una petición PUT a /categoria/1
Entonces el sistema debe retornar código 200 OK
Y el nombre debe actualizarse
Y la descripción debe mantener su valor original

Escenario 3: Categoría no encontrada

Dado que soy un administrador autenticado
Y no existe una categoría con id 999
Cuando envío una petición PUT a /categoria/999
Entonces el sistema debe retornar código 404 Not Found

Escenario 4: Error de validación

Dado que soy un administrador autenticado
Y el campo nombre tiene más de 100 caracteres
Cuando envío una petición PUT a /categoria/1
Entonces el sistema debe retornar código 400 Bad Request

Datos de Entrada (Request)

```
{  
  "nombre": "string (opcional, 2-100 caracteres si se envía)",  
  "descripcion": "string (opcional, max 500 caracteres si se envía)"  
}
```

Reglas de Negocio

ID	Regla
RN-004-01	Solo se actualizan los campos enviados (actualización parcial)
RN-004-02	Los campos nulos no modifican el valor existente
RN-004-03	El campo updatedAt se actualiza automáticamente
RN-004-04	El campo version se incrementa (optimistic locking)

Tareas Técnicas

- [] Crear DTO `CategoriaEdit` con validaciones opcionales
- [] Implementar método `applyTo()` en `CategoriaEdit`
- [] Implementar método `update()` en `CategoriaService`
- [] Implementar endpoint PUT /{id} en `CategoriaController`

HU-005: Eliminar Categoría

Información General

Campo	Valor
ID	HU-005
Epica	EP-01: Gestion de Categorías
Título	Eliminar Categoría (Soft Delete)
Prioridad	Media
Puntos de Historia	2

| ****Sprint**** | 1 |

Historia de Usuario

****Como**** Administrador del sistema

****Quiero**** poder eliminar una categoría

****Para**** mantener el catalogo limpio de categorías obsoletas

Criterios de Aceptación

Escenario 1: Eliminación exitosa

Dado que soy un administrador autenticado

Y existe una categoría con id 1 activa

Cuando envio una peticion DELETE a /categoria/1

Entonces el sistema debe retornar código 204 No Content

Y la categoría debe marcarse como eliminada (eliminado = true)

Y la categoría no debe aparecer en listados posteriores

Y la categoría debe permanecer en la base de datos

Escenario 2: Categoría no encontrada

Dado que soy un administrador autenticado

Y no existe una categoría con id 999

Cuando envio una peticion DELETE a /categoria/999

Entonces el sistema debe retornar código 404 Not Found

Escenario 3: Categoría ya eliminada

Dado que soy un administrador autenticado

Y existe una categoría con id 1 ya eliminada

Cuando envio una peticion DELETE a /categoria/1

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

| ID | Regla |

| ---- | ----- |

- | RN-005-01 | La eliminacion es logica (soft delete) |
- | RN-005-02 | El campo eliminado se establece en true |
- | RN-005-03 | El registro permanece en la base de datos |
- | RN-005-04 | Los productos asociados NO se eliminan automaticamente |

Tareas Técnicas

- [] Implementar método `deleteById()` en BaseRepository
- [] Implementar método `deleteById()` en `CategoriaService`
- [] Implementar endpoint DELETE /{id} en `CategoriaController`

EP-02: Gestion de Usuarios

HU-006: Registrar Usuario

Información General

- | Campo | Valor |
- |-----|-----|
- | **ID** | HU-006 |
- | **Epica** | EP-02: Gestion de Usuarios |
- | **Título** | Registrar Usuario |
- | **Prioridad** | Crítica |
- | **Puntos de Historia** | 5 |
- | **Sprint** | 1 |

Historia de Usuario

- Como** Visitante del sistema
- Quiero** poder registrarme como usuario
- Para** poder realizar compras y gestionar mis pedidos

Descripción Detallada

Los visitantes deben poder crear una cuenta en el sistema proporcionando sus datos personales. La contraseña debe almacenarse de forma segura usando encriptación BCrypt. El email debe ser único en el sistema.

Criterios de Aceptación

Escenario 1: Registro exitoso

Dado que soy un visitante no registrado

Y proporciono datos validos:

nombre	"Juan"	
apellido	"Perez"	
mail	"juan@email.com"	
celular	"1234567890"	
password	"miPassword123"	

Cuando envio una petición POST a /usuario

Entonces el sistema debe retornar código 201 Created

Y el cuerpo debe contener los datos del usuario (sin password)

Y el rol debe ser USUARIO por defecto

Y la contraseña debe almacenarse encriptada con BCrypt

Escenario 2: Error por email duplicado

Dado que ya existe un usuario con email "juan@email.com"

Cuando intento registrarme con el mismo email

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "Ya existe un usuario con el email: juan@email.com"

Escenario 3: Error por email invalido

Dado que proporciono un email con formato invalido "email-invalido"

Cuando envio una petición POST a /usuario

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar error de formato de email

Escenario 4: Error por password muy corto

Dado que proporciono una contraseña de menos de 6 caracteres

Cuando envío una petición POST a /usuario

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "La contraseña debe tener al menos 6 caracteres"

Escenario 5: Error por campos obligatorios vacíos

Dado que no proporciono el campo nombre

Cuando envío una petición POST a /usuario

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El nombre es obligatorio"

Datos de Entrada (Request)

```
{  
  "nombre": "string (obligatorio, 2-50 caracteres)",  
  "apellido": "string (obligatorio, 2-50 caracteres)",  
  "email": "string (obligatorio, formato email, unico)",  
  "celular": "string (opcional, máx 20 caracteres)",  
  "password": "string (obligatorio, min 6 caracteres)"  
}
```

Datos de Salida (Response)

```
{  
  "id": 1,  
  "nombre": "Juan",  
  "apellido": "Perez",  
  "mail": "juan@email.com",  
  "celular": "1234567890",  
  "rol": "USUARIO"  
}
```

Reglas de Negocio

ID	Regla
----	-----
RN-006-01	El nombre es obligatorio (2-50 caracteres)
RN-006-02	El apellido es obligatorio (2-50 caracteres)
RN-006-03	El email es obligatorio y debe ser único
RN-006-04	El email debe tener formato valido
RN-006-05	La contraseña es obligatoria (min 6 caracteres)
RN-006-06	La contraseña se encripta con BCrypt
RN-006-07	El rol por defecto es USUARIO
RN-006-08	La contraseña nunca se retorna en las respuestas

Tareas Técnicas

- [] Crear entidad `Usuario` con todos los campos
- [] Crear enum `Rol` (ADMIN, USUARIO)
- [] Crear DTO `UsuarioCreate` con validaciones
- [] Crear DTO `UsuarioDto` (sin campo password)
- [] Implementar `PasswordEncoder` con BCrypt
- [] Implementar validación de email unico en servicio
- [] Implementar método `save()` en `UsuarioService`
- [] Implementar endpoint POST en `UsuarioController`

Consideraciones de Seguridad

La contraseña debe hasheararse antes de persistir

Usar BCryptPasswordEncoder de Spring Security

El DTO de respuesta NO debe incluir la contraseña

Validar formato de email para prevenir inyecciones

HU-007: Listar Usuarios

Información General

Campo	Valor
-----	-----
ID	HU-007
Epica	EP-02: Gestion de Usuarios
Título	Listar Usuarios
Prioridad	Media
Puntos de Historia	2
Sprint	1

Historia de Usuario

****Como**** Administrador del sistema

****Quiero**** ver todos los usuarios registrados

****Para**** gestionar y administrar las cuentas del sistema

Criterios de Aceptación

Escenario 1: Listar usuarios existentes

Dado que soy un administrador autenticado

Y existen usuarios activos en el sistema

Cuando envio una peticion GET a /usuario

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array de usuarios

Y cada usuario NO debe incluir el campo password

Y no debe incluir usuarios eliminados

Escenario 2: Lista vacía

Dado que soy un administrador autenticado

Y no existen usuarios activos (excepto el admin)

Cuando envio una peticion GET a /usuario

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener al menos el usuario admin

Reglas de Negocio

ID	Regla
----	-----
RN-007-01	Solo administradores pueden listar usuarios
RN-007-02	La respuesta nunca incluye contraseñas
RN-007-03	Solo se muestran usuarios con eliminado = false

Tareas Técnicas

- [] Implementar método `findAll()` en `UsuarioService`
- [] Implementar endpoint GET en `UsuarioController`
- [] Asegurar que UsuarioDto no exponga password

HU-008: Obtener Usuario por ID

Información General

Campo	Valor
-----	-----
ID	HU-008
Epica	EP-02: Gestion de Usuarios
Título	Obtener Usuario por ID
Prioridad	Media
Puntos de Historia	2
Sprint	1

Historia de Usuario

- Como** Administrador del sistema
- Quiero** obtener los detalles de un usuario específico
- Para** ver la información completa de la cuenta

Criterios de Aceptación

Escenario 1: Usuario encontrado

Dado que soy un administrador autenticado

Y existe un usuario con id 1 activo

Cuando envio una peticion GET a /usuario/1

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener los datos del usuario (sin password)

Escenario 2: Usuario no encontrado

Dado que soy un administrador autenticado

Y no existe un usuario con id 999

Cuando envio una peticion GET a /usuario/999

Entonces el sistema debe retornar código 404 Not Found

Tareas Técnicas

- [] Implementar método `findById()` en `UsuarioService`
- [] Implementar endpoint GET /{id} en `UsuarioController`

HU-009: Actualizar Usuario

Información General

Campo	Valor
-----	-----
ID	HU-009
Epica	EP-02: Gestion de Usuarios
Título	Actualizar Usuario
Prioridad	Media
Puntos de Historia	5
Sprint	2

Historia de Usuario

****Como**** Usuario del sistema

****Quiero**** poder actualizar mis datos personales

****Para**** mantener mi información actualizada

Criterios de Aceptación

Escenario 1: Actualización exitosa de datos básicos

Dado que soy un usuario autenticado

Y quiero actualizar mi nombre y celular

Cuando envío una petición PUT a /usuario/{miId}

Entonces el sistema debe retornar código 200 OK

Y mis datos deben actualizarse correctamente

Escenario 2: Actualización de email con email unico

Dado que soy un usuario autenticado

Y quiero cambiar mi email a uno nuevo no registrado

Cuando envío una petición PUT a /usuario/{miId}

Entonces el sistema debe retornar código 200 OK

Y mi email debe actualizarse

Escenario 3: Error al cambiar email a uno existente

Dado que soy un usuario autenticado

Y quiero cambiar mi email a uno ya registrado por otro usuario

Cuando envío una petición PUT a /usuario/{miId}

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar que el email ya existe

Escenario 4: Actualización de contraseña

Dado que soy un usuario autenticado

Y quiero cambiar mi contraseña

Cuando envío una petición PUT con nuevo password

Entonces el sistema debe retornar código 200 OK

Y la nueva contraseña debe encriptarse con BCrypt

Datos de Entrada (Request)

```
{  
  "nombre": "string (opcional)",  
  "apellido": "string (opcional)",  
  "email": "string (opcional, debe ser único)",  
  "celular": "string (opcional)",  
  "password": "string (opcional, min 6 caracteres)"  
}
```

Reglas de Negocio

ID	Regla
----	-----
RN-009-01	Solo se actualizan los campos enviados
RN-009-02	Si se cambia el email, debe ser único
RN-009-03	Si se cambia la contraseña, debe encriptarse
RN-009-04	El campo updatedAt se actualiza automáticamente

Tareas Técnicas

- [] Crear DTO `UsuarioEdit` con validaciones opcionales
- [] Implementar validación de email unico en update
- [] Implementar método `update()` en `UsuarioService`
- [] Implementar endpoint PUT /{id} en `UsuarioController`

HU-010: Eliminar Usuario

Información General

Campo	Valor
-----	-----
ID	HU-010

****Epica****	EP-02: Gestion de Usuarios
****Título****	Eliminar Usuario (Soft Delete)
****Prioridad****	Baja
****Puntos de Historia****	2
****Sprint****	2

Historia de Usuario

****Como**** Administrador del sistema

****Quiero**** poder eliminar cuentas de usuario

****Para**** gestionar usuarios inactivos o problemáticos

Criterios de Aceptación

Escenario 1: Eliminación exitosa

Dado que soy un administrador autenticado

Y existe un usuario con id 2 activo

Cuando envio una petición DELETE a /usuario/2

Entonces el sistema debe retornar codigo 204 No Content

Y el usuario debe marcarse como eliminado

Y los pedidos del usuario deben permanecer en el sistema

Escenario 2: Usuario no encontrado

Dado que soy un administrador autenticado

Y no existe un usuario con id 999

Cuando envio una petición DELETE a /usuario/999

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

ID	Regla
RN-010-01	La eliminacion es logica (soft delete)
RN-010-02	Los pedidos del usuario NO se eliminan

| RN-010-03 | El usuario no puede autenticarse después de eliminado |

Tareas Técnicas

[] Implementar método `deleteById()` en `UsuarioService`

[] Implementar endpoint DELETE `/id` en `UsuarioController`

EP-03: Gestión de Productos

HU-011: Crear Producto

Información General

Campo	Valor
-----	-----
ID	HU-011
Epica	EP-03: Gestión de Productos
Título	Crear Producto
Prioridad	Crítica
Puntos de Historia	5
Sprint	2

Historia de Usuario

Como Administrador del sistema

Quiero poder crear nuevos productos en el catálogo

Para ofrecer nuevos artículos a los clientes

Descripción Detallada

El administrador necesita agregar productos al catálogo con toda su información relevante: nombre, precio, descripción, stock inicial, imagen y categoría. Cada producto debe estar asociado a una categoría existente.

Criterios de Aceptación

Escenario 1: Creación exitosa de producto

Dado que soy un administrador autenticado

Y existe una categoría con id 1

Y proporciono datos validos:

nombre	"Laptop Gaming Pro"	
precio	1599.99	
descripción	"Laptop de alto rendimiento"	
stock	25	
imagen	"laptop.jpg"	
disponible	true	
idCategoria	1	

Cuando envio una petición POST a /producto

Entonces el sistema debe retornar código 201 Created

Y el cuerpo debe contener el producto con su categoría

Escenario 2: Error por categoría inexistente

Dado que soy un administrador autenticado

Y no existe una categoría con id 999

Cuando intento crear un producto con idCategoria 999

Entonces el sistema debe retornar código 404 Not Found

Y el mensaje debe indicar que la categoría no existe

Escenario 3: Error por precio invalido

Dado que soy un administrador autenticado

Y proporciono un precio de 0 o negativo

Cuando envio una petición POST a /producto

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El precio debe ser mayor a 0"

Escenario 4: Error por stock negativo

Dado que soy un administrador autenticado

Y proporciono un stock negativo

Cuando envío una petición POST a /producto

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El stock no puede ser negativo"

Escenario 5: Producto creado con disponible = false

Dado que soy un administrador autenticado

Y proporciono disponible = false

Cuando envío una petición POST a /producto

Entonces el sistema debe retornar código 201 Created

Y el producto debe tener disponible = false

Datos de Entrada (Request)

```
{  
  "nombre": "string (obligatorio, 2-100 caracteres)",  
  "precio": "decimal (obligatorio, > 0.01)",  
  "descripcion": "string (opcional, máx 500 caracteres)",  
  "stock": "integer (obligatorio, >= 0)",  
  "imagen": "string (opcional)",  
  "disponible": "boolean (opcional, default: true)",  
  "idCategoria": "long (obligatorio)"  
}
```

Datos de Salida (Response)

```
{  
  "id": 1,  
  "nombre": "Laptop Gaming Pro",  
  "precio": 1599.99,  
  "descripción": "Laptop de alto rendimiento",  
  "stock": 25,  
  "imagen": "laptop.jpg",  
}
```

```
"disponible": true,  
  
"categoria": {  
  
  "id": 1,  
  
  "nombre": "Electrónica",  
  
  "descripcion": "Productos electrónicos"  
  
}  
}
```

Reglas de Negocio

ID	Regla
----	-----
RN-011-01	El nombre es obligatorio (2-100 caracteres)
RN-011-02	El precio es obligatorio y mayor a 0.01
RN-011-03	El stock debe ser >= 0
RN-011-04	La categoria es obligatoria y debe existir
RN-011-05	Si no se especifica disponible, es true por defecto
RN-011-06	El precio se almacena como BigDecimal

Tareas Técnicas

- [] Crear entidad `Producto` con todos los campos
- [] Crear relación ManyToOne con Categoria (LAZY)
- [] Crear DTO `ProductoCreate` con validaciones
- [] Crear DTO `ProductoDto` con categoría anidada
- [] Implementar método `save()` en `ProductoService`
- [] Validar existencia de categoría en servicio
- [] Implementar endpoint POST en `ProductoController`

HU-012: Listar Productos

Información General


```
| Campo | Valor |  
|-----|-----|  
| **ID** | HU-012 |  
| **Epica** | EP-03: Gestión de Productos |  
| **Título** | Listar Productos |  
| **Prioridad** | Alta |  
| **Puntos de Historia** | 2 |  
| **Sprint** | 2 |
```

Historia de Usuario

Como Usuario del sistema

Quiero ver todos los productos disponibles

Para explorar el catálogo y seleccionar productos para comprar

Criterios de Aceptación

Escenario 1: Listar productos existentes

Dado que existen productos activos en el sistema

Cuando envío una petición GET a /producto

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array de productos

Y cada producto debe incluir su categoría

Y no debe incluir productos eliminados

Escenario 2: Lista vacía

Dado que no existen productos activos en el sistema

Cuando envío una petición GET a /producto

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array vacío []

Tareas Técnicas

[] Implementar método `findAll()` en `ProductoService`

[] Implementar endpoint GET en `ProductoController`

[] Incluir categoria en la respuesta (DTO anidado)

HU-013: Obtener Producto por ID

Información General

Campo	Valor
-----	-----
ID	HU-013
Epica	EP-03: Gestión de Productos
Título	Obtener Producto por ID
Prioridad	Alta
Puntos de Historia	2
Sprint	2

Historia de Usuario

Como Usuario del sistema

Quiero ver los detalles de un producto específico

Para obtener información completa antes de comprarlo

Criterios de Aceptación

Escenario 1: Producto encontrado

Dado que existe un producto con id 1 activo

Cuando envio una petition GET a /producto/1

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener todos los datos del producto

Y debe incluir la información de la categoría

Escenario 2: Producto no encontrado

Dado que no existe un producto con id 999

Cuando envio una petition GET a /producto/999

Entonces el sistema debe retornar código 404 Not Found

Tareas Técnicas

[] Implementar método `findById()` en `ProductoService`

[] Implementar endpoint GET `/id` en `ProductoController`

HU-014: Listar Productos por Categoría

Información General

Campo	Valor
-----	-----
ID	HU-014
Epica	EP-03: Gestión de Productos
Título	Listar Productos por Categoría
Prioridad	Alta
Puntos de Historia	3
Sprint	2

Historia de Usuario

Como Usuario del sistema

Quiero filtrar productos por categoría

Para encontrar rápidamente los productos que me interesan

Criterios de Aceptación

Escenario 1: Productos encontrados en categoría

Dado que existe una categoría con id 1

Y existen productos activos en esa categoría

Cuando envío una petición GET a `/producto/categoría/1`

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener solo productos de esa categoría

Escenario 2: Categoría sin productos

Dado que existe una categoría con id 2

Y no existen productos activos en esa categoría

Cuando envío una petición GET a /producto/categoria/2

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array vacío []

Escenario 3: Categoría inexistente

Dado que no existe una categoría con id 999

Cuando envío una petición GET a /producto/categoria/999

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

ID	Regla
RN-014-01	La categoria debe existir
RN-014-02	Solo se retornan productos con eliminado = false
RN-014-03	Se incluyen productos disponibles e indisponibles

Tareas Técnicas

- [] Crear query `findAllByCategorialdAndEliminadoFalse()` en ProductoRepository
- [] Implementar método `findByCategoriald()` en `ProductoService`
- [] Validar existencia de categoría
- [] Implementar endpoint GET /categoria/{id} en `ProductoController`

HU-015: Actualizar Producto

Información General

Campo	Valor
ID	HU-015
Epica	EP-03: Gestión de Productos

****Título****	Actualizar Producto
****Prioridad****	Media
****Puntos de Historia****	5
****Sprint****	2

Historia de Usuario

****Como**** Administrador del sistema

****Quiero**** poder modificar los datos de un producto

****Para**** mantener actualizada la información del catálogo

Criterios de Aceptación

Escenario 1: Actualización completa exitosa

Dado que soy un administrador autenticado

Y existe un producto con id 1

Cuando envío datos válidos para actualizar

Entonces el sistema debe retornar código 200 OK

Y los datos deben actualizarse correctamente

Escenario 2: Actualización parcial (solo precio)

Dado que soy un administrador autenticado

Y existe un producto con id 1 con precio 100

Y envío solo el campo precio = 150

Cuando envío una petición PUT a /producto/1

Entonces el precio debe actualizarse a 150

Y los demás campos deben mantener sus valores

Escenario 3: Cambio de categoría

Dado que soy un administrador autenticado

Y existe un producto con id 1 en categoría 1

Y existe una categoría con id 2

Cuando envío idCategoria = 2

Entonces el producto debe cambiar a la categoría 2

Escenario 4: Error por categoría inexistente

Dado que soy un administrador autenticado

Y no existe una categoría con id 999

Cuando intento cambiar un producto a idCategoria 999

Entonces el sistema debe retornar código 404 Not Found

Escenario 5: Marcar producto como no disponible

Dado que soy un administrador autenticado

Y existe un producto con id 1 disponible = true

Cuando envío disponible = false

Entonces el producto debe marcarse como no disponible

Datos de Entrada (Request)

```
{  
  "nombre": "string (opcional)",  
  "precio": "decimal (opcional, > 0.01)",  
  "descripcion": "string (opcional)",  
  "stock": "integer (opcional, >= 0)",  
  "imagen": "string (opcional)",  
  "disponible": "boolean (opcional)",  
  "idCategoria": "long (opcional)"  
}
```

Reglas de Negocio

ID	Regla
----	-----
RN-015-01	Solo se actualizan los campos enviados
RN-015-02	Si se cambia categoria, debe existir
RN-015-03	El precio debe ser > 0.01 si se actualiza

| RN-015-04 | El stock debe ser ≥ 0 si se actualiza |

Tareas Técnicas

- [] Crear DTO `ProductoEdit` con validaciones opcionales
- [] Implementar método `applyTo()` en `ProductoEdit`
- [] Implementar método `update()` en `ProductoService`
- [] Implementar endpoint PUT `/id` en `ProductoController`

HU-016: Eliminar Producto

Información General

Campo	Valor
-----	-----
ID	HU-016
Epica	EP-03: Gestión de Productos
Título	Eliminar Producto (Soft Delete)
Prioridad	Media
Puntos de Historia	2
Sprint	2

Historia de Usuario

- Como** Administrador del sistema
- Quiero** poder eliminar productos del catálogo
- Para** remover productos obsoletos o descontinuados

Criterios de Aceptación

Escenario 1: Eliminación exitosa

Dado que soy un administrador autenticado

Y existe un producto con id 1 activo

Cuando envío una petición DELETE a `/producto/1`

Entonces el sistema debe retornar código 204 No Content

Y el producto debe marcarse como eliminado

Y el producto no debe aparecer en listados

Escenario 2: Producto no encontrado

Dado que soy un administrador autenticado

Y no existe un producto con id 999

Cuando envío una petición DELETE a /producto/999

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

ID	Regla
----	-------

----	-----
------	-------

RN-016-01	La eliminacion es logica (soft delete)
-----------	--

RN-016-02	Los detalles de pedido históricos mantienen referencia
-----------	--

Tareas Técnicas

[] Implementar método `deleteById()` en `ProductoService`

[] Implementar endpoint DELETE /{id} en `ProductoController`

EP-04: Gestión de Pedidos

HU-017: Crear Pedido

Información General

Campo	Valor
-------	-------

-----	-----
-------	-------

ID	HU-017
-----------	--------

Epica	EP-04: Gestión de Pedidos
--------------	---------------------------

Título	Crear Pedido
---------------	--------------

Prioridad	Crítica
------------------	---------

Puntos de Historia	8
---------------------------	---

| ****Sprint**** | 3 |

Historia de Usuario

****Como**** Usuario registrado

****Quiero**** poder crear un pedido con los productos seleccionados

****Para**** realizar una compra en el sistema

Descripción Detallada

El usuario debe poder crear un pedido especificando los productos y cantidades deseadas. El sistema debe validar disponibilidad y stock, calcular totales automáticamente, y reducir el stock de los productos.

Criterios de Aceptación

Escenario 1: Creación exitosa de pedido

Dado que soy un usuario autenticado con id 1

Y existe un producto con id 1, precio 100, stock 10, disponible = true

Y existe un producto con id 2, precio 50, stock 20, disponible = true

Cuando creo un pedido con:

| estado | PENDIENTE |

| formaPago | TARJETA |

| detalle 1 | producto 1, cantidad 2 |

| detalle 2 | producto 2, cantidad 3 |

Entonces el sistema debe retornar código 201 Created

Y el total debe ser $(100*2) + (50*3) = 350$

Y cada detalle debe tener su subtotal calculado

Y el stock del producto 1 debe reducirse de 10 a 8

Y el stock del producto 2 debe reducirse de 20 a 17

Y la fecha debe ser la fecha actual

Escenario 2: Error por usuario inexistente

Dado que no existe un usuario con id 999

Cuando intento crear un pedido con idUsuario 999

Entonces el sistema debe retornar código 404 Not Found

Y el mensaje debe indicar "Entidad con id 999 no encontrado"

Escenario 3: Error por producto no disponible

Dado que soy un usuario autenticado

Y existe un producto con id 1 y disponible = false

Cuando intento crear un pedido con ese producto

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El producto 'X' no está disponible para la venta"

Escenario 4: Error por stock insuficiente

Dado que soy un usuario autenticado

Y existe un producto con id 1, stock = 5

Cuando intento crear un pedido con cantidad = 10

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "Stock insuficiente para 'X'. Disponible: 5, Solicitado: 10"

Y el stock NO debe modificarse (rollback)

Escenario 5: Error por detalle vacío

Dado que soy un usuario autenticado

Cuando intento crear un pedido sin detalles

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar que se requiere al menos un detalle

Escenario 6: Error por estado no especificado

Dado que soy un usuario autenticado

Cuando intento crear un pedido sin especificar estado

Entonces el sistema debe retornar código 400 Bad Request

Y el mensaje debe indicar "El estado es obligatorio"

Escenario 7: Error por forma de pago no especificada

Dado que soy un usuario autenticado

Cuando intento crear un pedido sin especificar formaPago
Entonces el sistema debe retornar código 400 Bad Request
Y el mensaje debe indicar "La forma de pago es obligatoria"

Datos de Entrada (Request)

```
{  
  "estado": "PENDIENTE | CONFIRMADO | TERMINADO | CANCELADO",  
  "formaPago": "TARJETA | TRANSFERENCIA | EFECTIVO",  
  "idUsuario": "long (obligatorio)",  
  "detallePedido": [  
    {  
      "idProducto": "long (obligatorio)",  
      "cantidad": "integer (obligatorio, >= 1)"  
    }  
  ]  
}
```

Datos de Salida (Response)

```
{  
  "id": 1,  
  "fecha": "2024-12-10",  
  "estado": "PENDIENTE",  
  "total": 350.00,  
  "formaPago": "TARJETA",  
  "idUsuario": 1,  
  "detalles": [  
    {  
      "id": 1,  
      "cantidad": 2,
```

```
"subtotal": 200.00,  
  
"producto": {  
  
  "id": 1,  
  
  "nombre": "Producto A",  
  
  "precio": 100.00,  
  
  "stock": 8,  
  
  ...  
}  
  
}  
  
]  
  
}
```

Reglas de Negocio

ID	Regla
----	-----
RN-017-01	El usuario debe existir
RN-017-02	El estado es obligatorio
RN-017-03	La forma de pago es obligatoria
RN-017-04	Debe haber al menos un detalle de pedido
RN-017-05	Cada producto debe existir
RN-017-06	Cada producto debe estar disponible (disponible = true)
RN-017-07	Cada producto debe tener stock suficiente
RN-017-08	La cantidad de cada detalle debe ser ≥ 1
RN-017-09	El subtotal se calcula: $\text{precio} * \text{cantidad}$
RN-017-10	El total se calcula: suma de todos los subtotales
RN-017-11	La fecha se establece automáticamente
RN-017-12	El stock se reduce al crear el pedido
RN-017-13	Si alguna validación falla, se hace rollback completo

Tareas Técnicas

- [] Crear entidad `Pedido` con todos los campos
- [] Crear entidad `DetallePedido` con relaciones
- [] Crear enum `Estado` con todos los valores
- [] Crear enum `FormaPago` con todos los valores
- [] Crear relación ManyToOne Pedido -> Usuario
- [] Crear relación OneToMany Pedido -> DetallePedido (cascade)
- [] Crear relación ManyToOne DetallePedido -> Producto
- [] Crear DTO `PedidoCreate` con validaciones
- [] Crear DTO `DetallePedidoCreate` con validaciones
- [] Crear DTO `PedidoDto` con detalles anidados
- [] Crear DTO `DetallePedidoDto` con producto anidado
- [] Implementar métodos `tieneStockSuficiente()` y `reducirStock()` en Producto
- [] Implementar método `save()` en `PedidoService` con toda la logica
- [] Implementar endpoint POST en `PedidoController`
- [] Configurar transacción para rollback en caso de error
- [] Escribir tests para todos los escenarios

Consideraciones Técnicas

- Usar `@Transactional` para asegurar atomicidad
- Cargar productos una sola vez y usar mapa en memoria
- Validar todos los productos antes de modificar stock
- Calcular totales en el proceso de creación

HU-018: Listar Pedidos

Información General

```
| Campo | Valor |  
|-----|-----|  
| **ID** | HU-018 |  
| **Epica** | EP-04: Gestión de Pedidos |  
| **Título** | Listar Pedidos |  
| **Prioridad** | Alta |  
| **Puntos de Historia** | 2 |  
| **Sprint** | 3 |
```

Historia de Usuario

Como Administrador del sistema

Quiero ver todos los pedidos del sistema

Para gestionar y dar seguimiento a las órdenes

Criterios de Aceptación

Escenario 1: Listar pedidos existentes

Dado que soy un administrador autenticado

Y existen pedidos activos en el sistema

Cuando envío una petición GET a /pedido

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array de pedidos

Y cada pedido debe incluir sus detalles

Y no debe incluir pedidos eliminados

Escenario 2: Lista vacía

Dado que soy un administrador autenticado

Y no existen pedidos activos en el sistema

Cuando envío una petición GET a /pedido

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array vacío []

Tareas Técnicas

[] Implementar método `findAll()` en `PedidoService`

[] Implementar endpoint GET en `PedidoController`

[] Incluir detalles y productos en la respuesta

HU-019: Obtener Pedido por ID

Información General

Campo	Valor
-----	-----
ID	HU-019
Epica	EP-04: Gestión de Pedidos
Título	Obtener Pedido por ID
Prioridad	Alta
Puntos de Historia	2
Sprint	3

Historia de Usuario

Como Usuario del sistema

Quiero ver los detalles de un pedido específico

Para conocer el estado y contenido de mi orden

Criterios de Aceptación

Escenario 1: Pedido encontrado

Dado que existe un pedido con id 1 activo

Cuando envío una petición GET a /pedido/1

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener todos los datos del pedido

Y debe incluir los detalles con productos

Escenario 2: Pedido no encontrado

Dado que no existe un pedido con id 999

Cuando envío una petición GET a /pedido/999

Entonces el sistema debe retornar código 404 Not Found

Tareas Técnicas

[] Implementar método `findById()` en `PedidoService`

[] Implementar endpoint GET /{id} en `PedidoController`

HU-020: Listar Pedidos por Usuario

Información General

Campo	Valor
-----	-----
ID	HU-020
Epica	EP-04: Gestión de Pedidos
Título	Listar Pedidos por Usuario
Prioridad	Alta
Puntos de Historia	3
Sprint	3

Historia de Usuario

Como Usuario registrado

Quiero ver el historial de mis pedidos

Para hacer seguimiento de mis compras

Criterios de Aceptación

Escenario 1: Pedidos encontrados para el usuario

Dado que soy un usuario autenticado con id 1

Y tengo pedidos activos en el sistema

Cuando envío una petición GET a /pedido/usuario/1

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener solo mis pedidos

Y cada pedido debe incluir sus detalles

Escenario 2: Usuario sin pedidos

Dado que soy un usuario autenticado con id 2

Y no tengo pedidos en el sistema

Cuando envío una petición GET a /pedido/usuario/2

Entonces el sistema debe retornar código 200 OK

Y el cuerpo debe contener un array vacío []

Escenario 3: Usuario inexistente

Dado que no existe un usuario con id 999

Cuando envío una petición GET a /pedido/usuario/999

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

ID	Regla
----	-------

----	-----
------	-------

RN-020-01	El usuario debe existir
-----------	-------------------------

RN-020-02	Solo se retornan pedidos con eliminado = false
-----------	--

RN-020-03	Solo se retornan pedidos del usuario especificado
-----------	---

Tareas Técnicas

[] Crear query `findAllByUsuarioidAndEliminadoFalse()` en PedidoRepository

[] Implementar método `findByUsuarioid()` en `PedidoService`

[] Validar existencia del usuario

[] Implementar endpoint GET /usuario/{id} en `PedidoController`

HU-021: Actualizar Estado de Pedido

Información General

Campo	Valor
-------	-------

```
|-----|-----|  
  
| **ID** | HU-021 |  
  
| **Epica** | EP-04: Gestión de Pedidos |  
  
| **Título** | Actualizar Estado de Pedido |  
  
| **Prioridad** | Alta |  
  
| **Puntos de Historia** | 3 |  
  
| **Sprint** | 3 |
```

Historia de Usuario

Como Administrador del sistema

Quiero poder actualizar el estado y forma de pago de un pedido

Para reflejar el progreso del pedido en el sistema

Criterios de Aceptación

Escenario 1: Actualización de estado exitosa

Dado que soy un administrador autenticado
Y existe un pedido con id 1 en estado PENDIENTE
Cuando actualizo el estado a CONFIRMADO
Entonces el sistema debe retornar código 200 OK
Y el estado debe actualizarse a CONFIRMADO
Y el campo updatedAt debe actualizarse

Escenario 2: Actualización de forma de pago

Dado que soy un administrador autenticado
Y existe un pedido con id 1 con formaPago TARJETA
Cuando actualizo formaPago a TRANSFERENCIA
Entonces el sistema debe retornar código 200 OK
Y la forma de pago debe actualizarse

Escenario 3: Actualización parcial

Dado que soy un administrador autenticado

Y existe un pedido con id 1

Y envío solo el campo estado

Cuando envío una petición PUT a /pedido/1

Entonces solo el estado debe actualizarse

Y la forma de pago debe mantener su valor

Escenario 4: Pedido no encontrado

Dado que soy un administrador autenticado

Y no existe un pedido con id 999

Cuando envío una petición PUT a /pedido/999

Entonces el sistema debe retornar código 404 Not Found

Datos de Entrada (Request)

```
{  
  "estado": "PENDIENTE | CONFIRMADO | EN_PREPARACION | ENVIADO | ENTREGADO | TERMINADO |  
  CANCELADO (opcional)",  
  "formaPago": "TARJETA | TRANSFERENCIA | EFECTIVO (opcional)"  
}
```

Reglas de Negocio

ID	Regla
RN-021-01	Solo se actualizan estado y formaPago
RN-021-02	Los detalles del pedido NO se pueden modificar
RN-021-03	El total NO se recalcula
RN-021-04	Campos nulos no modifican el valor existente

Tareas Técnicas

- [] Crear DTO `PedidoEdit` con campos estado y formaPago
- [] Implementar método `applyTo()` en PedidoEdit
- [] Implementar método `update()` en `PedidoService`

[] Implementar endpoint PUT /{id} en `PedidoController`

HU-022: Eliminar Pedido

Información General

Campo	Valor
-----	-----
ID	HU-022
Epica	EP-04: Gestión de Pedidos
Título	Eliminar Pedido (Soft Delete)
Prioridad	Baja
Puntos de Historia	2
Sprint	3

Historia de Usuario

Como Administrador del sistema

Quiero poder eliminar pedidos

Para gestionar órdenes canceladas o erroneas

Criterios de Aceptación

Escenario 1: Eliminación exitosa

Dado que soy un administrador autenticado

Y existe un pedido con id 1 activo

Cuando envio una petición DELETE a /pedido/1

Entonces el sistema debe retornar código 204 No Content

Y el pedido debe marcarse como eliminado

Y los detalles deben mantenerse en la base de datos

Escenario 2: Pedido no encontrado

Dado que soy un administrador autenticado

Y no existe un pedido con id 999

Cuando envío una petición DELETE a /pedido/999

Entonces el sistema debe retornar código 404 Not Found

Reglas de Negocio

ID	Regla
----	-----
RN-022-01	La eliminacion es logica (soft delete)
RN-022-02	Los detalles del pedido NO se eliminan
RN-022-03	El stock de productos NO se restaura

Tareas Técnicas

[] Implementar método `deleteById()` en `PedidoService`

[] Implementar endpoint DELETE /{id} en `PedidoController`

EP-05: Infraestructura y Arquitectura

HU-023: Implementar Entidad Base

Información General

Campo	Valor
-----	-----
ID	HU-023
Epica	EP-05: Infraestructura
Título	Implementar Entidad Base
Prioridad	Crítica
Puntos de Historia	5
Sprint	1

Historia de Usuario

Como Desarrollador del sistema

Quiero tener una clase base para todas las entidades

****Para**** centralizar campos comunes y evitar duplicación de código

Descripción Técnica

Crear una clase abstracta `Base` que proporcione:

ID autogenerado

Campo eliminado para soft delete

Campos de auditoria (createdAt, updatedAt)

Campo versión para optimistic locking

Criterios de Aceptación

Escenario 1: Todas las entidades heredan de Base

Dado que tengo las entidades Categoria, Usuario, Producto, Pedido, DetallePedido

Entonces todas deben extender de la clase Base

Y todas deben tener los campos id, eliminado, createdAt, updatedAt, version

Escenario 2: ID autogenerado

Dado que creo una nueva entidad

Cuando la persisto en la base de datos

Entonces el ID debe generarse automáticamente

Escenario 3: Auditoria automática

Dado que creo una nueva entidad

Cuando la persisto en la base de datos

Entonces createdAt debe tener la fecha/hora actual

Y updatedAt debe tener la fecha/hora actual

Escenario 4: Actualización de timestamp

Dado que actualizo una entidad existente

Cuando la persisto en la base de datos

Entonces updatedAt debe actualizarse a la fecha/hora actual

Y createdAt debe mantener su valor original

Escenario 5: Optimistic locking

Dado que actualizo una entidad existente

Cuando la persisto en la base de datos

Entonces el campo versión debe incrementarse

Tareas Técnicas

- [] Crear clase abstracta `Base` con @MappedSuperclass
- [] Agregar campo id con @Id y @GeneratedValue(IDENTITY)
- [] Agregar campo eliminado con @Builder.Default = false
- [] Agregar campo createdAt con @CreationTimestamp
- [] Agregar campo updatedAt con @UpdateTimestamp
- [] Agregar campo versión con @Version
- [] Usar @SuperBuilder para herencia de builders
- [] Hacer que todas las entidades extiendan de Base

HU-024: Implementar Repositorio Base

Información General

Campo	Valor
-----	-----
ID	HU-024
Epica	EP-05: Infraestructura
Título	Implementar Repositorio Base
Prioridad	Crítica
Puntos de Historia	5
Sprint	1

Historia de Usuario

Como Desarrollador del sistema

Quiero tener un repositorio base con operaciones comunes

****Para**** centralizar la lógica de soft delete y queries comunes

Descripción Técnica

Crear una interfaz `BaseRepository<E extends Base, ID>` que:

Extienda de `JpaRepository`

Override `findAll()` para filtrar eliminados

Proporcione `findByIdOrThrow()` con excepción

Implemente `deleteById()` como soft delete

Criterios de Aceptación

Escenario 1: `findAll` filtra eliminados

Dado que existen entidades con `eliminado = true`

Cuando llamo a `findAll()`

Entonces solo debe retornar entidades con `eliminado = false`

Escenario 2: `findByIdOrThrow` retorna entidad

Dado que existe una entidad con `id 1` y `eliminado = false`

Cuando llamo a `findByIdOrThrow(1)`

Entonces debe retornar la entidad

Escenario 3: `findByIdOrThrow` lanza excepción

Dado que no existe una entidad con `id 999`

Cuando llamo a `findByIdOrThrow(999)`

Entonces debe lanzar `ResourceNotFoundException`

Escenario 4: `deleteById` hace soft delete

Dado que existe una entidad con `id 1` y `eliminado = false`

Cuando llamo a `deleteById(1)`

Entonces la entidad debe tener `eliminado = true`

Y la entidad debe permanecer en la base de datos

Tareas Técnicas

- [] Crear interfaz BaseRepository con @NoRepositoryBean
- [] Definir método findAllByEliminadoFalse()
- [] Definir método findByIdAndEliminadoFalse()
- [] Implementar método default findAll() que use findAllByEliminadoFalse()
- [] Implementar método default findByIdOrThrow()
- [] Implementar método default deleteById() con soft delete
- [] Marcar deleteById con @Transactional y @Modifying

HU-025: Implementar Manejo Global de Excepciones

Información General

Campo	Valor
-----	-----
ID	HU-025
Epica	EP-05: Infraestructura
Título	Implementar Manejo Global de Excepciones
Prioridad	Crítica
Puntos de Historia	5
Sprint	1

Historia de Usuario

- Como** Desarrollador del sistema
- Quiero** tener un manejador centralizado de excepciones
- Para** proporcionar respuestas de error consistentes en toda la API

Descripción Técnica

Crear `GlobalExceptionHandler` con @RestControllerAdvice que maneje:

ResourceNotFoundException -> 404

BusinessException -> 400

MethodArgumentNotValidException -> 400 con detalle de campos

IllegalStateException -> 409

Exception generica -> 500

Criterios de Aceptación

Escenario 1: ResourceNotFoundException retorna 404

Dado que ocurre una ResourceNotFoundException

Cuando el sistema maneja la excepción

Entonces debe retornar código 404 Not Found

Y el cuerpo debe tener formato ErrorResponse

Escenario 2: BusinessException retorna 400

Dado que ocurre una BusinessException

Cuando el sistema maneja la excepción

Entonces debe retornar código 400 Bad Request

Y el cuerpo debe tener formato ErrorResponse

Escenario 3: Validación retorna 400 con detalles

Dado que ocurre una MethodArgumentNotValidException

Cuando el sistema maneja la excepción

Entonces debe retornar código 400 Bad Request

Y el cuerpo debe tener formato ValidationErrorResponse

Y debe incluir el detalle de cada campo con error

Escenario 4: Exception genérica retorna 500

Dado que ocurre una excepción no controlada

Cuando el sistema maneja la excepción

Entonces debe retornar código 500 Internal Server Error

Y debe registrar el error en el log

Tareas Técnicas

[] Crear clase BusinessException

- [] Crear clase ResourceNotFoundException
- [] Crear record ErrorResponse (status, message, timestamp)
- [] Crear record ValidationErrorResponse (status, message, timestamp, errors)
- [] Crear GlobalExceptionHandler con @RestControllerAdvice
- [] Implementar @ExceptionHandler para cada tipo de excepción
- [] Agregar logging con @Slf4j

HU-026: Implementar Encriptación de Contraseñas

Información General

Campo	Valor
-----	-----
ID	HU-026
Epica	EP-05: Infraestructura
Título	Implementar Encriptación de Contraseñas
Prioridad	Crítica
Puntos de Historia	3
Sprint	1

Historia de Usuario

- Como** Arquitecto de seguridad
- Quiero** que las contraseñas se almacenen encriptadas
- Para** proteger la información sensible de los usuarios

Descripción Técnica

Implementar componente `PasswordEncoder` que use BCrypt para:

Encriptar contraseñas nuevas

Comparar contraseñas en autenticación

Criterios de Aceptación

Escenario 1: Encriptación de contraseña

Dado que tengo una contraseña en texto plano "password123"

Cuando la encripto con PasswordEncoder

Entonces debe retornar un hash BCrypt

Y el hash debe ser diferente cada vez (salt aleatorio)

Escenario 2: Verificación de contraseña correcta

Dado que tengo un hash BCrypt de "password123"

Cuando verifico con la contraseña "password123"

Entonces debe retornar true

Escenario 3: Verificación de contraseña incorrecta

Dado que tengo un hash BCrypt de "password123"

Cuando verifico con la contraseña "otraPassword"

Entonces debe retornar false

Tareas Técnicas

- [] Agregar dependencia spring-security-crypto
- [] Crear clase PasswordEncoder como @Component
- [] Implementar método encode(String password)
- [] Implementar metodo matches(String raw, String encoded)
- [] Usar BCryptPasswordEncoder internamente

HU-027: Implementar Carga Inicial de Datos

Información General

Campo	Valor
-----	-----
ID	HU-027
Epica	EP-05: Infraestructura
Título	Implementar Carga Inicial de Datos

| ****Prioridad**** | Media |

| ****Puntos de Historia**** | 2 |

| ****Sprint**** | 1 |

Historia de Usuario

****Como**** Administrador del sistema

****Quiero**** que exista un usuario administrador por defecto

****Para**** poder acceder al sistema en la primera ejecución

Descripción Técnica

Crear componente `UserLoad` que implemente CommandLineRunner y:

Se ejecute al iniciar la aplicación

Verifique si existen usuarios

Cree un usuario admin por defecto si no existen

Criterios de Aceptación

Escenario 1: Crear admin en base de datos vacía

Dado que la base de datos está vacía

Cuando la aplicación inicia

Entonces debe crear un usuario administrador

Con email "admin@admin.com"

Y password "123456" (encriptado)

Y rol ADMIN

Escenario 2: No crear admin si ya existen usuarios

Dado que ya existen usuarios en la base de datos

Cuando la aplicación inicia

Entonces NO debe crear el usuario administrador

Tareas Técnicas

[] Crear clase UserLoad que implemente CommandLineRunner

[] Inyectar UsuarioRepository y PasswordEncoder

[] Verificar count() de usuarios

[] Crear usuario admin si count == 0

[] Registrar en log la creación

HU-028: Configurar Documentación OpenAPI

Información General

Campo	Valor
-----	-----
ID	HU-028
Epica	EP-05: Infraestructura
Título	Configurar Documentación OpenAPI
Prioridad	Media
Puntos de Historia	2
Sprint	1

Historia de Usuario

Como Desarrollador consumidor de la API

Quiero tener documentación interactiva de los endpoints

Para entender y probar fácilmente la API

Descripción Técnica

Configurar SpringDoc OpenAPI para generar documentación automática con Swagger UI.

Criterios de Aceptación

Escenario 1: Acceso a Swagger UI

Dado que la aplicación está corriendo

Cuando accedo a /swagger-ui/index.html

Entonces debe mostrar la interfaz de Swagger

Y debe listar todos los endpoints

Escenario 2: Acceso a especificación OpenAPI

Dado que la aplicación está corriendo

Cuando accedo a /api-docs

Entonces debe retornar la especificacion OpenAPI en JSON

Tareas Técnicas

[] Agregar dependencia springdoc-openapi-starter-webmvc-ui

[] Configurar propiedades en application.properties

[] Verificar que todos los endpoints aparecen documentados

HU-029: Configurar CORS

Información General

Campo	Valor
-----	-----
ID	HU-029
Epica	EP-05: Infraestructura
Título	Configurar CORS
Prioridad	Alta
Puntos de Historia	1
Sprint	1

Historia de Usuario

Como Desarrollador frontend

Quiero que la API permita peticiones desde otros orígenes

Para poder consumir la API desde aplicaciones web

Criterios de Aceptación

Escenario 1: Petición desde otro origen

Dado que hago una petición desde `http://localhost:3000`

Cuando la API procesa la petición

Entonces debe incluir headers CORS apropiados

Y la petición debe ser exitosa

Tareas Técnicas

[] Agregar `@CrossOrigin("*")` en todos los controladores

[] Considerar configuración más restrictiva para producción

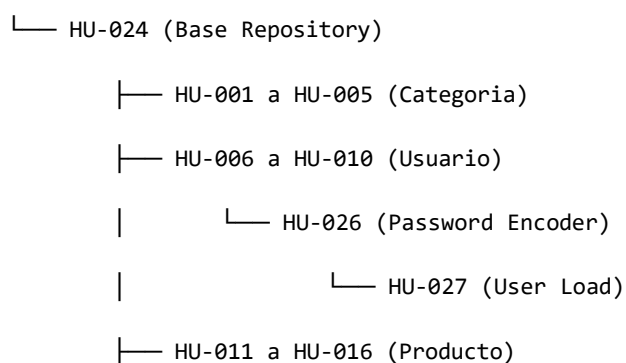
Matriz de Trazabilidad

Historias de Usuario por Epica

Epica	Historias	Total HU	Story Points
-----	-----	-----	-----
EP-01: Categorías	HU-001 a HU-005	5	12
EP-02: Usuarios	HU-006 a HU-010	5	16
EP-03: Productos	HU-011 a HU-016	6	19
EP-04: Pedidos	HU-017 a HU-022	6	20
EP-05: Infraestructura	HU-023 a HU-029	7	23
TOTAL		**29**	**90**

Dependencias entre Historias

HU-023 (Base Entity)




```

|           └─ Dependencia de Categoría
└─ HU-017 a HU-022 (Pedido)
      └─ Dependencia de Usuario y Producto

HU-025 (Exception Handler)
      └─ Usado por todas las HU de endpoints
  
```

Priorización del Backlog

Sprint 1 - Fundamentos (Semana 1-2)

	Prioridad	Historia	Story Points	Descripción
1	5	HU-023	Entidad Base	
2	5	HU-024	Repositorio Base	
3	5	HU-025	Manejo de Excepciones	
4	3	HU-026	Encriptacion	
5	3	HU-001	Crear Categoria	
6	2	HU-002	Listar Categorías	
7	2	HU-003	Obtener Categoria	
8	3	HU-004	Actualizar Categoria	
9	2	HU-005	Eliminar Categoria	
10	5	HU-006	Registrar Usuario	
11	2	HU-027	Carga Inicial	
12	2	HU-028	OpenAPI	
13	1	HU-029	CORS	
Total Sprint 1		**40**		

Sprint 2 - Usuarios y Productos (Semana 3-4)

	Prioridad	Historia	Story Points	Descripción
--	-----------	----------	--------------	-------------

1	HU-007	2	Listar Usuarios	
2	HU-008	2	Obtener Usuario	
3	HU-009	5	Actualizar Usuario	
4	HU-010	2	Eliminar Usuario	
5	HU-011	5	Crear Producto	
6	HU-012	2	Listar Productos	
7	HU-013	2	Obtener Producto	
8	HU-014	3	Productos por Categoría	
9	HU-015	5	Actualizar Producto	
10	HU-016	2	Eliminar Producto	
Total Sprint 2		**30**		

Sprint 3 - Pedidos (Semana 5-6)

Prioridad	Historia	Story Points	Descripción	
-----	-----	-----	-----	
1	HU-017	8	Crear Pedido	
2	HU-018	2	Listar Pedidos	
3	HU-019	2	Obtener Pedido	
4	HU-020	3	Pedidos por Usuario	
5	HU-021	3	Actualizar Pedido	
6	HU-022	2	Eliminar Pedido	
Total Sprint 3		**20**		

Definición de Completado (DoD)

Una Historia de Usuario se considera **COMPLETADA** cuando:

Código

- ☐ El código fuente está escrito y sigue los estándares del proyecto
- ☐ El código compila sin errores ni warnings
- ☐ Se utilizan los patrones de diseño definidos (DTO, Service, Repository)
- ☐ Se implementan todas las validaciones específicas
- ☐ Se manejan correctamente las excepciones

Testing

- ☐ Tests unitarios escritos con cobertura $\geq 80\%$
- ☐ Tests de integración para endpoints
- ☐ Todos los tests pasan exitosamente
- ☐ Se probaron todos los escenarios de los criterios de aceptación

Documentación

- ☐ Endpoint documentado automáticamente en Swagger
- ☐ Código comentado donde sea necesario
- ☐ README actualizado si aplica

Revisión

- ☐ Code review aprobado por al menos un compañero
- ☐ No hay deuda técnica identificada sin documentar
- ☐ El código está integrado en la rama principal

Funcionalidad

- ☐ La funcionalidad cumple todos los criterios de aceptación
- ☐ No hay regresiones en funcionalidades existentes

[] La API retorna los códigos HTTP correctos